Frontispiece

TRAINING MANUAL

Coding With //Screen-Readers

CONFIDENTIAL — For Internal Use Only

Copyright © 2026 Michael Ryan Hunsaker, M.Ed., Ph.D. Coding With Screen-Readers is licensed under a Creative Commons Attribution-NoNCommerical-NoDerivatives 4.0 International: https://creativecommons.org/licenses/by-nc-nd/4.0/ All rights reserved. NO AI TRAINING: Without in any way limiting the author's rights under copyright, any use of this publication to "train" generative artificial intelligence (AI) technologies to generate text is expressly prohibited.

Contents

Teacher's Guide: Introduction to Computer Science with JavaScript 3
Unit 0: Setting Up Your JavaScript Development Environment 37
Unit 1: Foundations — Getting Started with JavaScript 39
Unit 2: Numbers, Input, and the Math Object 43
Unit 3: Control Flow — Decisions and Loops 46
Unit 4: Data Structures — Arrays and Objects 50
Unit 5: Strings and String Methods 56
Unit 6: Functions 57
Unit 7: Classes and Object-Oriented Programming 60
Unit 8: Files, Modules, and Error Handling 64
Unit 9: Modules, Collections, and Testing 68
Unit 10: Project-Based Learning 73
Unit 11: Capstone Project — Personal Data Manager in JavaScript 78

Dedication

To the countless individuals who have been told they lack the capability to code, yet remain determined to learn and create through accessible coding practices, this book is dedicated.

Epigraph

The power of the Web is in its universality. Access by everyone regardless of disability is an essential aspect

— Tim Berners-Lee, Inventor of the World Wide Web

Introduction to Computer Science with JavaScript

For Visually Impaired Students

Course Overview

This curriculum introduces computer science concepts using JavaScript, focusing on Node.js for server-side/terminal execution. All exercises are text-based and terminal-focused — no browser required for core concepts.

Target Audience: Students with no prior programming experience using screen readers (NVDA, JAWS, VoiceOver, or Windows Narrator).

Why JavaScript?

JavaScript is one of the most widely used programming languages in the world. It powers:

- Web development (front-end and back-end)
- Server applications (Node.js)
- Mobile apps (React Native)
- Desktop apps (Electron)

Learning JavaScript opens doors to a vast ecosystem of tools and opportunities.

Tools Required

- **Node.js 18 LTS+** — https://nodejs.org/
- **Visual Studio Code** with the following extensions:
  - JavaScript (ES6) code snippets
  - ESLint
  - Prettier
- A screen reader (NVDA, JAWS, or VoiceOver)

Accessibility Notes

- All program output is text-based (Node.js terminal/REPL).
- JavaScript uses curly braces `{}` for code blocks, like Java.
- VS Code's Screen Reader Optimized Mode works well with JavaScript.
- Use `Ctrl + Shift + P` -> "Toggle Screen Reader Optimized" in VS Code.
- Use `F8` / `Shift+F8` to navigate between errors.

Units

Unit	Title
Unit 0	Setting Up Your JavaScript Development Environment
Unit 1	Foundations — Getting Started with JavaScript
Unit 2	Numbers, Input, and the Math Object
Unit 3	Control Flow — Decisions and Loops
Unit 4	Data Structures — Arrays and Objects
Unit 5	User Input in Node.js
Unit 6	Functions
Unit 7	Classes and Object-Oriented Programming
Unit 8	Files and Error Handling
Unit 9	Modules, Collections, and Testing
Unit 10	Project-Based Learning
Unit 11	Capstone Project

References

- MDN Web Docs: https://developer.mozilla.org/en-US/docs/Web/JavaScript
- Eloquent JavaScript (free online): https://eloquentjavascript.net/
- Node.js Documentation: https://nodejs.org/docs/

Teacher's Guide: Introduction to Computer Science with JavaScript

For Students with Visual Impairments — Complete Instructor Edition

---

Table of Contents

1. Philosophy and Why JavaScript
2. JavaScript-Specific Accessibility Challenges
3. Classroom Setup and Configuration
4. Spoken JavaScript Conventions
5. Unit 0–1: Setup and Foundations
6. Unit 2: Numbers, Input, and Math
7. Unit 3: Control Flow
8. Unit 4: Arrays and Objects
9. Unit 5: Strings
10. Unit 6: Functions
11. Unit 7: Classes and OOP
12. Unit 8: Files, Modules, and Error Handling
13. Unit 9: Collections, Analysis, and Testing
14. Units 10–11: Projects and Capstone
15. Bridging from Python/Java to JavaScript
16. Assessment Strategies
17. Supporting Diverse Needs
18. Alternative and Supplementary Resources
19. Quick Reference Sheets

---

1. Philosophy and Why JavaScript

The Case for Teaching JavaScript to Blind Students JavaScript has a unique profile of benefits and challenges for visually impaired learners:

Universal relevance. JavaScript is the only language that runs natively in every web browser. A student who learns JavaScript can eventually build tools that improve their own accessibility experience — custom browser extensions, accessible widgets, automation scripts. This self-advocacy potential is meaningful.

Node.js enables accessible terminal programs. While JavaScript began in browsers, Node.js allows running JavaScript programs in the terminal — the same accessible text-based environment used for Python. This curriculum focuses on Node.js to keep all I/O in the terminal where screen readers excel.

Asynchronous programming is a real-world skill. JavaScript's `async/await` model reflects how most modern software actually works — programs that wait for I/O, network requests, and user events. Learning this early prepares students for professional development.

The npm ecosystem. Over 2 million packages on npm provide pre-built solutions for almost any problem. This is a double-edged concern for beginners (dependency complexity), but the ability to say `npm install` and gain powerful functionality is professionally relevant.

JSON is native. `JSON.parse()` and `JSON.stringify()` are built into JavaScript with no import required. For data persistence projects, this reduces friction.

An Honest Account of Challenges **JavaScript's permissiveness is a silent-failure risk.** Unlike Python (which raises exceptions for type mismatches) and Java (which rejects them at compile time), JavaScript silently converts types. `"5" + 3` = `"53"`. `"5" - 3` = `2`. `0 == false` = `true`. Without visual scanning of intermediate results, these silent failures can propagate through a program undetected. This must be addressed through defensive programming habits taught from day one.

Asynchronous programming is conceptually hard. The `async`/`await` pattern required for user input is a genuine hurdle for beginners. There is no way to eliminate it from a Node.js input-based curriculum. The approach in this guide is to scaffold it carefully and defer full explanation of the event loop to later units.

Multiple valid syntaxes. Function declarations, function expressions, arrow functions, methods — all do essentially the same thing. `var`, `let`, `const` — three variable declarations. `==` and `===` — two equality operators. This variety, which professionals call "JavaScript fatigue," creates unnecessary cognitive overhead for beginners. The approach in this curriculum is to teach one style consistently (`const`/`let`, arrow functions, `===`) and acknowledge alternatives exist without teaching them in parallel.

---

2. JavaScript-Specific Accessibility Challenges

Silent Type Coercion This is the greatest accessibility risk in JavaScript for blind students. Unlike Python's `TypeError` and Java's compile-time type check, JavaScript silently converts types in ways that can produce wrong answers with no error:

Expression	Result	Expected by Programmer	Problem
`"5" + 3`	`"53"`	`8`	`+` concatenates when one operand is a string
`"5" - 3`	`2`	`2`	`-` converts string to number
`"5" * "3"`	`15`	`15`	Multiplication converts both
`"" == false`	`true`	Maybe?	Loose equality coercion

Expression	Result	Expected by Programmer	Problem
`0 == false`	`true`	Maybe?	Same
`null == undefined`	`true`	Probably not	
`NaN === NaN`	`false`	`true`	NaN is not equal to itself

Mitigation: Require `Number()` before any arithmetic on user input. Require `===` always. Teach `typeof` and `isNaN()` as defensive checks. Make logging intermediate values an explicit exercise before running any calculation.

`undefined` vs. `null` vs. `NaN` vs. `false` vs. `0` vs. `""` All six of these are "falsy" in JavaScript. Aurally, they are indistinguishable unless you call `typeof` or check explicitly. A student who hears `console.log(result)` and the screen reader announces "false" cannot know whether `result` is `false`, `0`, `""`, `null`, `undefined`, or `NaN` without further inspection.

Teaching approach: Establish a diagnostic template:

```
console.log("Value:", result, "Type:", typeof result);
```

This outputs both the value and its type, providing audio context that `console.log(result)` alone does not.

Callback Hell and Asynchronous Confusion Before `async/await`, Node.js input used nested callbacks:

```
// Confusing nesting --- avoid in teaching
rl.question("Name? ", (name) => {
  rl.question("Age? ", (age) => {
    rl.question("City? ", (city) => {
      console.log(`${name}, ${age}, ${city}`);
    });
  });
});
```

This nesting produces code that is confusing to read AND to hear — the indentation deepens with each question. The `async/await` pattern linearizes this:

```
// Clear and linear --- use this exclusively
const name = await askQuestion("Name? ");
const age = await askQuestion("Age? ");
const city = await askQuestion("City? ");
console.log(`${name}, ${age}, ${city}`);
```

Teach `async/await` from the start and avoid callback patterns entirely in this course.

---

3. Classroom Setup and Configuration

VS Code for JavaScript **Required extensions:**

- **ESLint** — catches type issues, undefined variables, and style violations before running code
- **Prettier** — auto-formats code consistently (run on save with `editor.formatOnSave: true`)
- **JavaScript (ES6) code snippets** — tab-complete common patterns

Recommended ESLint configuration (`.eslintrc.json` in project root):

```
{
  "env": { "node": true, "es2021": true },
  "extends": "eslint:recommended",
  "rules": {
    "eqeqeq": "error",
    "no-unused-vars": "warn",
    "no-undef": "error",
    "prefer-const": "warn",
    "no-console": "off"
  }
}
```

The `eqeqeq: error` rule makes `==` a linting error — it must be `===`. This is an automated enforcement of the rule that prevents the most common silent bugs.

Node.js REPL accessibility:

```
node                    # Start REPL
.help                   # List REPL commands
.exit                   # Quit (or Ctrl+D)
```

The Node.js REPL is excellent for early exploration. Results are printed immediately after each expression, providing audio feedback without the compile-run-read cycle.

Alternative JavaScript Environments

Environment	Platform	Accessibility	Notes
Node.js + VS Code	All	High	Recommended setup

Environment	Platform	Accessibility	Notes
Node.js + terminal only	All	High	For students who prefer minimal UI
Replit.com	Web	Medium	No-install; test with student's setup
CodeSandbox	Web	Medium	Cloud editor; variable accessibility
GitHub Codespaces	Web	High	Cloud VS Code; same accessibility as local
Glitch.com	Web	Medium	Project-based; variable accessibility
Observable	Web	Low	Notebook-style; primarily visual
Deno	Runtime	High	Alternative to Node.js; similar accessibility

---

4. Spoken JavaScript Conventions

Symbol/Pattern	Spoken Form
`const`	"const"
`let`	"let"
`var`	"var" (avoid; legacy)
`===`	"triple equals" or "strict equal"
`!==`	"strict not equal"
`==`	"loose equal" (should hear this rarely)
`=>`	"arrow" or "fat arrow"
`...arr`	"spread arr"
`...args`	"rest args"
`async`	"async"
`await`	"await"
`undefined`	"undefined"
`null`	"null"
`NaN`	"NaN" (say each letter N-A-N)
`typeof`	"typeof"
`instanceof`	"instanceof"
`new`	"new"
`this`	"this"
`class`	"class"
`extends`	"extends"
`super`	"super"

Symbol/Pattern	Spoken Form
`import`	"import"
`export`	"export"
`require()`	"require"
`module.exports`	"module dot exports"
`Promise`	"Promise"
`async function`	"async function"
`() => {}`	"arrow function"
`(x) => x * 2`	"arrow function taking x, returning x times 2"
`#field`	"hash field" or "private field"
`` `template ${var}` ``	"backtick template literal with variable var"

Reading an arrow function:

```
const doubleIfPositive = (n) => (n > 0 ? n * 2 : n);
```

"const doubleIfPositive gets arrow function taking n, returning: if n greater than 0, n times 2, else n"

---

5. Unit 0–1: Setup and Foundations

Lesson 0.1: Installing Node.js

Instructional Cues **Verify installation at every step:**

```
node --version    # Should print v18.x.x or later
npm --version     # Should print a version number
npx --version     # Should print a version number (npx is bundled with npm)
```

All three must work. If `npm` is missing, the student may have installed only the Node.js runtime without npm. Reinstall from nodejs.org (npm is included in the standard installer).

REPL first — files second: Begin with `node` in the terminal. Let the student type expressions and hear results immediately. This builds confidence before introducing the file -> run -> read cycle:

```
$ node
> 5 + 3
8
> "Hello " + "World"
'Hello World'
> typeof 42
'number'
```

```
> typeof "hello"
'string'
> .exit
```

The npm init conversation: When initializing a project, `npm init -y` generates `package.json` automatically. Walk the student through reading the generated file:

```
npm init -y
cat package.json   # or type package.json on Windows
```

`package.json` is plain JSON text — fully readable with a screen reader. Understanding its structure is important for later lessons about dependencies and scripts.

Non-Obvious Accessibility Strategies

- **Node.js version managers:** `nvm` (Node Version Manager) allows multiple Node.js versions. For a student who may work on multiple machines or projects, `nvm use 18` is a consistent way to set the Node.js version. `nvm` is command-line-only and fully accessible.
- **Global vs. local packages:** `npm install -g` installs globally; `npm install` installs locally (in `node_modules`). Teach local installation as the default. Global packages can interfere across projects.

Thought Experiments

1. *"JavaScript was created in 10 days in 1995. What marks of that hasty design are still visible in modern JavaScript? (Think about `typeof null === 'object'`, `NaN !== NaN`, `0 == false`.) Why haven't these been fixed?"*
2. *"Node.js uses a single-threaded event loop to handle many concurrent operations. A traditional multi-threaded approach uses multiple threads running in parallel. For a web server handling thousands of requests, what are the tradeoffs of each approach? Which tends to be more memory efficient?"*
3. *"npm has millions of packages. The average Node.js project depends on hundreds of packages, many maintained by single developers with no organizational backing. The 'left-pad incident' in 2016 broke thousands of projects when one 11-line package was unpublished. What does this incident reveal about the risks and benefits of the npm ecosystem?"*
4. *"JavaScript runs in browsers, in Node.js servers, in mobile apps via React Native, in desktop apps via Electron, in IoT devices, and even in serverless cloud functions. What does this ubiquity tell you about the relationship between a language's design and its adoption?"*

Lesson 1.3: Variables and Data Types

Instructional Cues **`const` vs. `let` — establish the rule on day one:**

"Use `const` for everything. If you get an error saying you're trying to reassign a `const`, THEN change it to `let`. This approach produces cleaner code and prevents accidental reassignment bugs."

Demonstrate the error:

```
const x = 5;
x = 10; // TypeError: Assignment to constant variable.
```

"This error is helpful. `const` is protecting you from changing something you said was fixed."

Dynamic typing with auditory ambiguity:

Run this sequence in the REPL and ask the student to predict each output before pressing Enter:

```
typeof 42; // "number"
typeof "42"; // "string"
typeof true; // "boolean"
typeof null; // "object"  <- the famous bug
typeof undefined; // "undefined"
typeof {}; // "object"
typeof []; // "object"  <- arrays are objects!
typeof function () {}; // "function"
```

"Notice that both `{}` (object) and `[]` (array) produce 'object'. How do you tell them apart? `Array.isArray([])` returns `true`. `Array.isArray({})` returns `false`."

Template literals vs. concatenation:

```
const name = "Alice";
const age = 15;

// Concatenation --- confusing order of operations when read aloud
console.log("Hello, " + name + "! You are " + age + " years old.");

// Template literal --- linear, clean, easy to hear
console.log(`Hello, ${name}! You are ${age} years old.`);
```

Read both versions aloud with the student. Ask which is easier to follow aurally. Template literals are almost universally clearer when heard, because the variable values appear where they belong in the sentence.

Thought Experiments

1. *"JavaScript has only one number type for integers and decimals. Python has separate `int` and `float`. Java has `int`, `long`, `double`, `float`. What precision*

problems arise from having a single number type? When does `0.1 + 0.2 = 0.30000000000000004` matter, and when doesn't it?"

2. *"JavaScript has six falsy values: `false`, `0`, `""`, `null`, `undefined`, `NaN`. Everything else is truthy — including `"false"` (a non-empty string), `[]` (an empty array), and `{}` (an empty object). Design a quiz: ask the student whether ten different expressions are truthy or falsy. Which answers are surprising?"*

3. *"`var` has function scope; `let` and `const` have block scope. What does 'scope' mean? How does a variable declared inside an `if` block with `var` vs. `let` behave differently? Which behavior is more predictable and why?"*

---

6. Unit 2: Numbers, Input, and Math

Lesson 2.2: User Input in Node.js

Instructional Cues **The input helper module pattern:**

Provide the `input.js` module in every project skeleton. Students should not need to re-create this boilerplate:

```
// input.js
const readline = require("readline");
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(prompt) {
  return new Promise((resolve) => rl.question(prompt, resolve));
}

function close() {
  rl.close();
}

module.exports = { askQuestion, close };
```

Walk through what each line does at least once so students understand the structure, but do not require students to write it from scratch until they're comfortable with async programming.

The `async`/`await` mental model — teach this story:

"When your code hits `await`, imagine it puts up a 'please wait' sign and steps aside. Other things can happen while you wait. When the answer arrives, your code picks up exactly where it paused. The `async` keyword on the function declaration is your promise that 'this function may pause at some point.'"

```
async function main() {
  console.log("Starting..."); // 1: runs immediately
  const name = await askQuestion("Name? "); // 2: pauses here until user types
  console.log("Got name:", name); // 3: runs after user answers
  const age = await askQuestion("Age? "); // 4: pauses again
  console.log(`${name} is ${age}`); // 5: runs after user answers again
  close();
}

main(); // Kick off the async function
```

Ask: "What line is running while the program waits for user input?" (None — the function is paused. The event loop can handle other things.)

Why `main()` must be called at the bottom: `async function main()` defines the function but doesn't run it. The `main()` call at the end actually starts execution. Without the call, nothing happens. This trips up beginners — demonstrate it.

Non-Obvious Accessibility Strategies

- **`rl.close()` is mandatory:** Without `rl.close()`, the Node.js process continues running after `main()` finishes. The terminal appears to hang — cursor blinks but no prompt. For a screen reader user, this is confusing. Teach `close()` as the final step of every program that uses input.
- **Prompt trailing space:** Always include a trailing space in prompt strings: `"Enter your name: "`. Without the space, the cursor appears immediately after the colon with no separation, which some screen readers render as "Enter your name:cursor" — an awkward audio experience.
- **Input validation loop:** Without validation, a blank Enter or a non-numeric string for a number causes silent failure. Provide this reusable pattern:

```
async function getNumber(prompt) {
  while (true) {
    const input = await askQuestion(prompt);
    const num = Number(input.trim());
    if (!isNaN(num)) return num;
    console.log("Invalid input: please enter a number.");
  }
}
```

Teach this pattern and require students to use it whenever they need numeric input.

Thought Experiments

1. *"The `readline` module uses an 'event listener' pattern — `rl.question(prompt, callback)` means 'when the user answers, call this function.' The `async`/`await` pattern wraps this in a Promise. What problem does the Promise solve that the raw callback doesn't? (Think about reading the code linearly.)"*

2. *"Node.js uses a single-threaded event loop. While your program is waiting for user input (`await askQuestion`), the thread is free. What else could the thread be doing? In a real application, this might mean handling a different request or updating state. What does this tell you about why `async`/`await` is important for scalable programs?"*

3. *"Synchronous input (blocking until the user responds) and asynchronous input (doing other things while waiting) represent fundamentally different programming models. For a single-user command-line program, which matters? For a server handling 10,000 simultaneous users, which matters more? Why?"*

---

7. Unit 3: Control Flow

Lessons 3.1–3.3

Instructional Cues **`===` enforcement — make it automatic:**

With ESLint configured to flag `==`, the student will hear a warning whenever they use loose equality. Walk through the first warning together:

```
if (userInput == 5) {   // ESLint: Expected '===' and instead saw '=='
```

"ESLint is protecting you from a common silent bug. Change this to `===`. The only exception is checking for null-or-undefined: `x == null` is sometimes used as a shorthand for `x === null || x === undefined` — but avoid this until you understand it."

Truthy/falsy in input validation:

```
const name = await askQuestion("Enter your name: ");
if (!name.trim()) {
  console.log("Name cannot be empty.");
  // re-prompt...
}
```

`!name.trim()` is true when the trimmed string is empty (`""` is falsy). This is idiomatic JavaScript. Read it aloud: "if not name dot trim" — ask: "What does 'not an empty string' mean? Why is an empty string falsy?"

Ternary operator — limit its use:

The ternary `condition ? valueIfTrue : valueIfFalse` is concise but can be hard to follow when read aloud:

```
const status = score >= 60 ? "pass" : "fail";
```

Read: *"const status gets: if score greater-than-or-equal 60, quote pass, else quote fail"*

Teach it but establish this rule: "If the ternary is easy to read aloud in one breath, use it. If it requires re-reading, use if-else."

Thought Experiments

1. *"JavaScript's `==` performs type coercion before comparing. TypeScript (a typed superset of JavaScript) was partly created to address this. What does TypeScript's existence tell you about the real-world cost of JavaScript's permissiveness? What did developers gain from TypeScript that JavaScript didn't provide?"*

2. *"Consider: `if (userName)` vs. `if (userName !== '' && userName !== null && userName !== undefined)`. The first uses JavaScript's truthiness; the second is explicit. In a code review, which would you prefer seeing? What are the readability and correctness tradeoffs?"*

3. *"JavaScript's optional chaining (`?.`) and nullish coalescing (`??`) were added to the language to handle null/undefined more gracefully. `user?.address?.city` returns `undefined` instead of throwing if `user` or `address` is null. Why did this feature take decades to add to a language created in 1995?"*

4. *"The `switch` statement falls through by default in JavaScript. Modern JavaScript has been moving away from `switch` toward mapping objects for dispatch tables. `const actions = { 'add': addFunc, 'delete': deleteFunc }; actions[command]?.();`. What are the accessibility implications of this pattern vs. a `switch` statement when read with a screen reader?"*

---

8. Unit 4: Arrays and Objects

Lesson 4.1: Arrays and Functional Methods

Instructional Cues **Introduce `map`, `filter`, `reduce` as a trio:**

Don't teach them one per lesson — teach the relationship between them first:

"These three methods each take a function and apply it to every element in an array. `map` transforms elements. `filter` keeps or removes elements. `reduce` combines elements into one value. Together they cover most array processing."

Then give one shared example that uses all three:

```
const students = [
  { name: "Alice", score: 88 },
  { name: "Bob", score: 62 },
  { name: "Charlie", score: 95 },
  { name: "Diana", score: 71 },
];

// 1. Filter passing students (score >= 70)
const passing = students.filter((s) => s.score >= 70);

// 2. Map to get just names
const passingNames = passing.map((s) => s.name);

// 3. Reduce to build a report string
const report = passingNames.reduce(
  (acc, name) => acc + name + ", ",
  "Passing: ",
);

console.log(report); // "Passing: Alice, Charlie, Diana, "
console.log("Count:", passing.length);
```

Accessible printing for arrays of objects:

```
// Accessible: numbered, labeled fields
students.forEach((student, index) => {
  console.log(`${index + 1}. ${student.name}: ${student.score}`);
});

// Less accessible: raw object output
console.log(students); // Produces cluttered bracket notation when read aloud
```

Chaining — build up incrementally:

Show chaining as a composition of steps, each of which you can trace separately:

```
const result = numbers
  .filter((n) => n % 2 === 0) // Step 1: keep evens
  .map((n) => n ** 2) // Step 2: square each
  .reduce((sum, n) => sum + n, 0); // Step 3: sum

// DEBUG: break the chain to trace each step
const step1 = numbers.filter((n) => n % 2 === 0);
console.log("After filter:", step1);
const step2 = step1.map((n) => n ** 2);
console.log("After map:", step2);
const step3 = step2.reduce((sum, n) => sum + n, 0);
console.log("Final sum:", step3);
```

The debug version (step-by-step) is more accessible during development. Consolidate to the chain after confirming correctness.

Thought Experiments

1. *"`map`, `filter`, and `reduce` come from functional programming — a paradigm based on mathematical functions that have no side effects. JavaScript arrays also have `forEach` which is for side effects only. What is the philosophical distinction between `map` and `forEach`? Why does this distinction matter for reasoning about code correctness?"*

2. *"The `sort` method in JavaScript sorts in-place and mutates the original array. `filter`, `map`, and `reduce` return new arrays, leaving the original unchanged. Why the asymmetry? What bugs could arise from `sort`'s mutation? How would you sort without mutating: `[...arr].sort(...)`?"*

3. _"Functional composition: `const double = n => n _ 2; const addOne = n => n + 1; const doubleAndAdd = n => addOne(double(n))`. How does function composition scale? What is the advantage of composing small, focused functions over writing one large function that does everything?"*

4. *"The spread operator `[...arr]` creates a shallow copy of an array. For an array of primitives `[1, 2, 3]`, the spread copy is fully independent. For an array of objects `[{name: 'A'}, {name: 'B'}]`, the objects inside are still shared. What does 'shallow copy' mean? When does this distinction cause bugs?"*

Lesson 4.2: Objects

Instructional Cues **Destructuring — introduce before objects are complex:**

```
const person = { name: "Alice", age: 15, city: "Portland" };

// Without destructuring:
const name = person.name;
const age = person.age;

// With destructuring:
const { name, age } = person;

// In function parameter:
function introduce({ name, age }) {
  console.log(`Hi, I'm ${name}, ${age} years old.`);
}
introduce(person);
```

Destructuring is ubiquitous in modern JavaScript. Teaching it early reduces the verbosity of object access code, which in turn makes programs cleaner to read and hear.

`Object.keys()`, `Object.values()`, `Object.entries()` as navigation tools:

For a student who cannot visually inspect an object in a debugger panel, these methods provide audio-accessible object introspection:

```
const config = { theme: "dark", fontSize: 14, notifications: true };

// What properties does this object have?
console.log(Object.keys(config).join(", "));

// What are all the values?
Object.values(config).forEach((v) => console.log(v));

// Both together:
Object.entries(config).forEach(([key, value]) => {
  console.log(`${key}: ${value}`);
});
```

Optional chaining for safe nested access:

```
const user = { profile: { address: null } };

// Without optional chaining --- crashes if address is null
const city = user.profile.address.city; // TypeError!

// With optional chaining --- returns undefined instead
const city = user?.profile?.address?.city; // undefined, no crash

// With nullish coalescing for a default
const city = user?.profile?.address?.city ?? "Unknown city";
```

This pattern is essential for working with real-world data where fields may be missing.

Thought Experiments

1. *"JavaScript objects can have properties added or removed at any time. `user.newField = 'value'` just works. Java objects have a fixed set of fields defined at compile time. Which approach is better for representing data from an API that might add new fields in future versions? Which is better for ensuring all required fields are present?"*

2. *"`Object.freeze(obj)` makes an object immutable — you can't add, remove, or modify properties. This mimics the behavior of a Java `final` field or Python's `frozenset`. When would immutable objects be useful? What patterns become possible when you can guarantee an object won't change?"*

3. *"Prototype-based inheritance means JavaScript objects can inherit from other objects directly, without classes. Classes (introduced in ES6) are 'syntactic sugar' over this prototype system. What does this mean? What can you do with prototypes that you cannot easily express with class syntax?"*

4. *"JSON (JavaScript Object Notation) is the universal data exchange format. It looks almost like JavaScript object literals but has stricter rules: all keys must be double-quoted strings, no functions, no comments. Why must JSON be stricter than JavaScript object literals? What security problems would arise if JSON allowed functions?"*

---

9. Unit 5: Strings

Instructional Cues **Template literals for multiline strings:**

```
const menu = `
--- Main Menu ---
1. Add item
2. View all
3. Search
4. Delete
5. Quit
-----------------`;

console.log(menu);
```

Template literals with newlines are much cleaner for building multiline output than repeated `console.log()` calls. Teach this pattern for menus and reports.

Regular expression basics — connect to real need:

Before introducing regex syntax, establish the problem it solves:

"You have user input that might have extra spaces, uppercase, or punctuation. You want just the words. Without regex, you'd call `.trim().toLowerCase().replace(',', '').replace('.', '')...` for every punctuation mark. Regex lets you express 'any non-letter character' once."

```
const messy = "  Hello, World!   How are you?  ";
const words = messy
  .trim()
  .toLowerCase()
  .split(/\W+/)
  .filter((w) => w.length > 0);
console.log(words); // ['hello', 'world', 'how', 'are', 'you']
```

Teach `\w+` (word characters), `\d+` (digits), `\s+` (whitespace), and the `g` and `i` flags. Don't attempt to teach full regex — a basic vocabulary is enough for data cleaning tasks.

Thought Experiments

1. *"Template literals use backticks `` ` `` while regular strings use `"` or `'`. Backticks on a keyboard are typically in the upper-left corner. For touch typists,*

backticks are less natural than quotes. What keyboard shortcuts or text expansions might a blind programmer use to reduce the friction of frequently-needed but awkward characters?"

2. *"String immutability in JavaScript (same as Python) means every* `.toUpperCase()`*,* `.replace()`*, and* `.slice()` *creates a new string. For a loop that builds a long string incrementally, what is the efficient alternative? (Hint: consider* `Array.push()` + `.join()` *as a form of StringBuilder.)"*

---

10. Unit 6: Functions

Extended Instructional Cues **The four function forms — teach one style, acknowledge the rest:**

```
// 1. Function declaration (hoisted --- can be called before declaration)
function add(a, b) {
  return a + b;
}

// 2. Function expression (not hoisted)
const multiply = function (a, b) {
  return a * b;
};

// 3. Arrow function --- SHORT BODY (implicit return)
const square = (n) => n * n;

// 4. Arrow function --- LONG BODY (explicit return required)
const absolute = (n) => {
  if (n < 0) return -n;
  return n;
};
```

Teach style 3 (short arrow) and style 4 (long arrow) as the primary forms. Acknowledge 1 and 2 exist and how to read them. Do not attempt to teach all four simultaneously.

The implicit return trap — a critical accessibility issue:

```
// These DO return a value:
const double = (n) => n * 2; // [x] implicit return
const greet = (name) => `Hi ${name}`; // [x] implicit return

// This does NOT return a value (the curly brace requires explicit return):
const broken = (n) => {
  n * 2; // No return statement! Returns undefined.
};

// Fixed:
```

```
const fixed = (n) => {
  return n * 2;
};
```

When heard aloud, `n => { n * 2; }` and `n => { return n * 2; }` sound different — the `return` keyword is present in one and absent in the other. Teach students to listen for `return` inside block-bodied arrow functions.

Higher-order functions — connect to previous array methods:

```
// map() takes a function as argument
const doubled = [1, 2, 3].map((n) => n * 2);

// We can also pass a named function
const double = (n) => n * 2;
const doubled2 = [1, 2, 3].map(double);

// This works because map expects a function --- it doesn't care if it's
// named or anonymous, arrow or traditional
```

"Every time you used `array.map(...)`, you were passing a function as an argument. Higher-order functions — functions that take other functions — are normal JavaScript."

Closures — establish the mental model before the code:

"A closure is a function that remembers the environment where it was created, even after that environment is gone. Imagine a letter in an envelope. The letter is the function. The envelope is the environment — it contains variables the function needs. Even if the person who wrote the letter is gone, the letter still has access to everything in its envelope."

```
function makeCounter(startValue = 0) {
  let count = startValue; // This is the "envelope"

  return {
    increment: () => ++count,
    decrement: () => --count,
    reset: () => {
      count = startValue;
    },
    getCount: () => count,
  };
}

const counter = makeCounter(10);
counter.increment();
counter.increment();
counter.increment();
console.log(counter.getCount()); // 13 --- count persists!
```

Thought Experiments

1. *"Arrow functions don't have their own `this` — they inherit `this` from the surrounding scope. Regular functions create their own `this`. This was a major source of bugs in pre-ES6 JavaScript. How many of the following do you think involved confusing `this`? (a) Typos, (b) Variable scope errors, (c) Async timing bugs, (d) `this` binding bugs. In JavaScript specifically, (d) is disproportionately common — why?"*

2. *"Pure functions always produce the same output for the same input and have no side effects. `Math.sin(x)` is pure. `console.log(x)` is not (it has a side effect). Why are pure functions easier to test? Why are pure functions easier to reason about in a screen-reader debugging session?"*

3. *"Memoization caches the results of function calls so repeated calls with the same arguments return the cached result. Write a description of a memoized `fibonacci(n)` function. What is saved? What is the tradeoff? When would memoization be harmful rather than helpful?"*

4. *"JavaScript is a multi-paradigm language: you can write imperative code (explicit step-by-step), object-oriented code (classes and methods), or functional code (pure functions and immutable data). For the accessible data manager project, which paradigm produces the most maintainable code? The most testable? The most readable when heard aloud?"*

---

11. Unit 7: Classes and OOP

Instructional Cues **`toString()` is mandatory:**

```
class BankAccount {
  #balance;

  constructor(owner, initialBalance = 0) {
    this.owner = owner;
    this.#balance = initialBalance;
  }

  toString() {
    return `BankAccount[owner=${this.owner}, balance=$${this.#balance.toFixed(2)}]`;
  }
}

const account = new BankAccount("Alice", 1000);
console.log(account.toString()); // BankAccount[owner=Alice, balance=$1000.00]
console.log(`${account}`); // Also calls toString() implicitly
```

Require `toString()` in every class. Explain: "When your screen reader reads `console.log(myAccount)`, it reads `toString()`. Without it, you

hear `[object Object]` — completely useless. Define `toString()` to hear meaningful information."

Private fields # — establish as the default:

```
class Dog {
  #name; // Private --- cannot be accessed outside the class
  #age;

  constructor(name, age) {
    this.#name = name;
    this.#age = age;
  }

  get name() {
    return this.#name;
  } // Getter --- controlled access
  get age() {
    return this.#age;
  }

  set age(value) {
    if (value < 0) throw new Error("Age cannot be negative");
    this.#age = value;
  }

  bark() {
    console.log(`${this.#name} says: Woof!`);
  }

  toString() {
    return `Dog[name=${this.#name}, age=${this.#age}]`;
  }
}
```

"Private fields starting with `#` are truly private — not just conventional privacy. `myDog.#name` outside the class throws a SyntaxError. This is real encapsulation."

Thought Experiments

1. *"JavaScript classes are 'syntactic sugar' over prototype-based inheritance. TypeScript adds static typing on top of JavaScript classes. With three layers (prototypes -> ES6 classes -> TypeScript classes), is JavaScript's OOP model more or less trustworthy than Java's, which has only one layer? What does 'syntactic sugar' imply about the behavior of the underlying mechanism?"*

2. *"The `class` keyword was added to JavaScript in 2015, 20 years after the language was created. In those 20 years, JavaScript developers built massive applications without classes using only prototypes and closures. What*

does this tell you about the role of syntax vs. capability in programming languages?"

3. *"Mixins are a way to compose behavior from multiple sources without inheritance. Instead of* `class A extends B`*, you copy methods from B onto A's prototype. JavaScript supports this; Java does not (without interfaces). When would mixins be more flexible than single inheritance? What problems could they cause?"*

---

12. Unit 8: Files, Modules, and Error Handling

Lesson 8.1: Files

Instructional Cues **`__dirname` and path construction:**

```
const path = require("path");
const fs = require("fs");

// __dirname is the directory of the current file
const dataFile = path.join(__dirname, "data", "contacts.json");

// Path joining prevents slash vs. backslash issues on Windows/macOS/Linux
console.log("Data file location:", dataFile);
```

For a student who cannot visually navigate a file tree, `console.log(__dirname)` and `console.log(dataFile)` are essential diagnostic tools. Teach them to print file paths as the first debugging step when a file isn't found.

`fs.existsSync()` before file operations:

```
if (fs.existsSync(dataFile)) {
  const data = fs.readFileSync(dataFile, "utf8");
  return JSON.parse(data);
} else {
  console.log("No data file found. Starting with empty data.");
  return [];
}
```

Always check existence before reading. The alternative — catching `ENOENT` in a try-catch — works but puts normal "first run" behavior in the error path.

Lesson 8.3: CommonJS Modules **Module design for accessibility:**

A well-designed module has a clear exports interface. Teach students to think of their module's exports as its "public API" — what other files can use:

```
// contacts.js --- exports only what other files need
const path = require("path");
const fs = require("fs");

const DATA_FILE = path.join(__dirname, "..", "data", "contacts.json");

function loadContacts() {
  /* ... */
}
function saveContacts(contacts) {
  /* ... */
}
function findContact(contacts, name) {
  /* ... */
}

// Export only the public interface
module.exports = { loadContacts, saveContacts, findContact };
```

"Everything not in `module.exports` is private to this file. Other files can use `loadContacts` and `saveContacts`, but they cannot directly access `DATA_FILE`. This is encapsulation at the module level."

Thought Experiments

1. *"Node.js's CommonJS (`require`) and the ES Module standard (`import`) are two different module systems in JavaScript. This creates confusion — some packages use one, some use the other. TypeScript can use both. Why did two module systems emerge? What does this history say about how standards develop in open-source ecosystems?"*

2. *"Synchronous file I/O (`readFileSync`) blocks execution until the file is read. Asynchronous file I/O (`readFile` or `fs.promises.readFile`) allows other code to run while waiting. For a startup script that reads configuration before doing anything else, which makes more sense? For a web server that handles thousands of requests, which matters more?"*

3. *"Custom error classes allow code to throw and catch specific, named errors: `throw new ValidationError('email', 'Invalid format')`. What is the advantage of this over throwing `new Error('Validation failed: email field invalid format')`? Think about code that catches and handles the error."*

---

13. Unit 9: Collections, Analysis, and Testing

Jest Testing

Instructional Cues **Jest output is accessible:**

Run `npx jest --verbose` for maximum accessibility. The `--verbose` flag produces:

```
PASS src/math.test.js
  add function
    [x] adds two positive numbers (3ms)
    [x] adds negative numbers (1ms)
    [x] handles zero (0ms)
  getGrade function
    [x] returns A for score >= 90 (0ms)
    [x] returns F for score < 60 (0ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
```

Each test name is announced. Failures include file path, test name, expected vs. received values, and a diff. All of this is plain text — fully screen-reader-accessible.

Add to `package.json` scripts:

```
"scripts": {
    "test": "jest --verbose",
    "test:watch": "jest --verbose --watch"
}
```

`npm test` runs all tests. `npm run test:watch` reruns tests whenever a file changes — useful during active development.

Test naming convention:

```
describe("BankAccount", () => {
  describe("deposit", () => {
    test("increases balance by deposited amount", () => {
      /* ... */
    });
    test("rejects negative amounts", () => {
      /* ... */
    });
    test("rejects zero amount", () => {
      /* ... */
    });
  });
  describe("withdraw", () => {
    test("decreases balance by withdrawn amount", () => {
      /* ... */
    });
    test("throws InsufficientFundsError when balance is too low", () => {
      /* ... */
```

```
    });
  });
});
```

The nested `describe` / `test` structure, when announced by Jest with `--verbose`, reads as complete sentences: "BankAccount > deposit > increases balance by deposited amount."

Thought Experiments

1. *"Jest's `expect(value).toEqual(expected)` performs deep comparison — it checks that two objects have the same structure and values, not that they are the same object in memory. Why is deep comparison usually what you want when testing? When would you specifically want reference equality (`toBe`)?"*

2. *"Snapshot testing in Jest captures the output of a component or function the first time, then checks future runs against that snapshot. What are the advantages? What are the risks? When might a snapshot become 'stale' and mislead you into thinking code is correct when it isn't?"*

3. *"Code coverage measures what percentage of your code is executed by tests. 100% coverage is a goal but not a guarantee of correctness. Design a function with 100% test coverage that still has a bug. (Hint: think about boundary conditions — what if the test covers the case where x=5 but not x=0?)"*

---

14. Units 10–11: Projects and Capstone

JavaScript Project Structure

```
my-data-manager/
|--- src/
|   |--- models/
|   |   +--- contact.js
|   |--- storage/
|   |   +--- dataManager.js
|   |--- ui/
|   |   +--- consoleUI.js
|   +--- utils/
|       |--- input.js          (the readline helper)
|       +--- validation.js     (reusable validation functions)
|--- data/
|   +--- contacts.json
|--- tests/
|   |--- models/
|   |   +--- contact.test.js
```

```
|   +--- storage/
|       +--- dataManager.test.js
|--- .eslintrc.json
|--- package.json
+--- index.js
```

`package.json` scripts — teach as accessibility tools:

```
{
  "scripts": {
    "start": "node index.js",
    "test": "jest --verbose",
    "test:watch": "jest --verbose --watch",
    "lint": "eslint src/",
    "lint:fix": "eslint src/ --fix"
  }
}
```

`npm start` — consistent command regardless of entry point filename. `npm test` — runs all tests. `npm run lint` — checks for errors without running the program.

`npm run lint` as a pre-run check: Teach students to run `npm run lint` before running the program. ESLint catches undefined variables, unused variables, and `==` before the program runs — much faster than finding runtime errors.

Accessible Output Design for Capstone Require students to design their program's output to be navigable and self-describing:

```
// Navigation landmark prints
function printSeparator(title) {
  const line = "-".repeat(40);
  console.log(line);
  if (title) console.log(`  ${title.toUpperCase()}`);
  console.log(line);
}

// Numbered list with clear field labels
function printContactList(contacts) {
  if (contacts.length === 0) {
    console.log("No contacts found.");
    return;
  }
  printSeparator("contacts");
  contacts.forEach((c, i) => {
    console.log(`${i + 1}. ${c.name}`);
    console.log(`   Phone: ${c.phone || "not provided"}`);
    console.log(`   Email: ${c.email || "not provided"}`);
  });
```

```
  console.log(`Total: ${contacts.length} contact(s)`);
}

// Confirmation messages --- every action must produce one
function confirmAction(action, details) {
  console.log(`[x] ${action}: ${details}`);
}
```

Thought Experiments

1. *"Your capstone is a command-line program. Eventually, it could become a web application. What would need to change? What would transfer directly? Write a two-column list: 'Can reuse as-is' and 'Must rewrite.'"*

2. *"The accessible design principle 'no visual-only cues for critical information' means every status, error, or confirmation must be communicated through text, not color or icon alone. Where in your program have you relied on visual formatting? How would you replace it?"*

3. *"Open-source your capstone project. Write a README that explains: what the program does, how to install it, how to run it, and specifically how to use it with a screen reader. What information would a sighted user find obvious but a blind user needs explicitly stated?"*

---

15. Bridging from Python/Java to JavaScript

From Python to JavaScript

Python	JavaScript	Key Difference
`print("hello")`	`console.log("hello")`	Method vs. function
`input("prompt")`	`await askQuestion("prompt")`	Async in JavaScript
`f"Hello {name}"`	`` `Hello ${name}` ``	f-string vs. template literal
`def func(x):`	`const func = x => ...`	Arrow function style
`True/False`	`true/false`	Lowercase in JavaScript
`None`	`null` and `undefined`	JS has two "nothing" values
`my_list`	`myList`	JavaScript uses camelCase
`len(x)`	`x.length`	Property, not function

Python	JavaScript	Key Difference
`x is None`	`x === null` or `x === undefined`	Different null checks
`import json`	`const fs = require('fs')`	Built-in `JSON` object, no import
`if __name__ == '__main__':`	No direct equivalent (just call `main()`)	

Thought experiments for Python->JS students:

1. *"In Python, you use spaces to define blocks (indentation). In JavaScript, you use `{}`. Python's blocks are invisible — they're just the alignment of characters. JavaScript's blocks are explicitly marked. Which feels more natural? Which provides clearer information when read aloud?"*

2. *"Python's `None` and JavaScript's `null`/`undefined` both represent 'nothing,' but they behave differently. Python has one 'nothing'; JavaScript has two. When is `null` appropriate? When is `undefined` appropriate? Under what circumstances would you encounter one vs. the other?"*

3. *"Python's type system and JavaScript's both use dynamic typing, but JavaScript's is more permissive — it coerces types rather than raising errors. After using both, which approach do you find more helpful for learning? Which is more appropriate for production software?"*

---

16. Assessment Strategies

Alternative Assessments for JavaScript **Console output analysis:** Provide a JavaScript program and 3–4 sample inputs. Ask the student to predict what the program prints for each input before running it. This tests reading comprehension without requiring writing.

Linting-first review: Submit code that is free of all ESLint warnings. A student who achieves this has internalized JavaScript's best practices at the syntax level.

Refactoring exercises: Give a working but poorly structured program. Ask the student to refactor it to use functions, modules, or classes appropriately. Assess the quality of the architecture, not just whether it runs.

Test-first implementation: Give the student a set of failing tests. Ask them to write the implementation that makes the tests pass. This tests code comprehension and problem-solving without a blank-page start.

---

17. Supporting Diverse Needs

For Students Who Are New to Async Programming Async/await is conceptually hard. For students who struggle:

1. Provide the complete `input.js` helper and instruct them to use it without modifying it.
2. For the first several lessons, treat `async function main() { ... } main();` as boilerplate, not content.
3. Introduce the conceptual explanation of async/await only after students are comfortable with the rest of the language.
4. Consider deferring async until after basic functions are mastered.

For Students with Additional Cognitive Considerations

- ESLint provides immediate text feedback for common errors — use it as a learning tool. Hearing "no-unused-vars" and then explaining what that means is a teaching moment triggered by the student's own code.
- JavaScript's many syntactic variations can be overwhelming. Enforce a single consistent style (arrow functions, `const`/`let`, `===`) and defer alternatives.
- Use the REPL liberally for short experiments so the student gets immediate feedback without file management overhead.

---

18. Alternative and Supplementary Resources

Textbooks and Reading

Resource	Level	Format	Accessibility
Eloquent JavaScript (Haverbeke, free online)	Beginner–Intermediate	Free HTML	Excellent screen-reader-friendly HTML
You Don't Know JS (Simpson, free on GitHub)	Intermediate	Free HTML/PDF	In-depth on closures, scope, types
JavaScript: The Good Parts (Crockford)	Intermediate	Print	Short; focuses on the reliable subset of JS
JavaScript: The Definitive Guide (Flanagan)	Reference	Print/ebook	Comprehensive; good for looking up specifics

Resource	Level	Format	Accessibility
MDN Web Docs (developer.mozilla.org)	Reference	Free HTML	Excellent HTML accessibility; definitive JS reference
Node.js Documentation (nodejs.org/docs)	Reference	Free HTML	Official Node.js API reference
JavaScript.info	Beginner–Advanced	Free HTML	Very well-organized; plain HTML; comprehensive

javascript.info is especially recommended — it's a comprehensive free tutorial organized from basics to advanced topics, all in clean, accessible HTML. It's often more readable than MDN for learning (vs. reference), and screen readers navigate it very well.

Online Courses and Platforms

Platform	Course	Accessibility	Notes
freeCodeCamp	JavaScript Algorithms & Data Structures	Medium	Certification path; some UI inaccessible
Codecademy	Learn JavaScript	Partial	Editor accessible; some UI elements not
Exercism.io	JavaScript Track	High	Terminal-based; mentor feedback
JavaScript30 (Wesbos)	30 vanilla JS projects	Medium	Video-based; transcripts available
edX	Various JavaScript courses	High	Video transcripts; accessible exercises
Scrimba	Learn JavaScript	Low	Interactive video; poor screen reader support
Khan Academy	JavaScript course	Medium	Browser-based editor; variable accessibility
NodeSchool.io	Node.js workshops	High	Command-line workshopper format; fully accessible

NodeSchool.io is especially recommended — workshops are run entirely in the terminal via a command-line tool. Students work through exercises, get feedback from the tool, and progress at their own pace. The terminal-first design makes it highly accessible.

Video and Audio Resources

Resource	Description	Accessibility
Traversy Media (YouTube)	JavaScript crash courses and projects	Captions available
The Net Ninja (YouTube)	In-depth JavaScript series	Captions available
Academind (YouTube)	Deep dives into JS/Node	Captions available
Kevin Powell (YouTube)	JavaScript and web fundamentals	Captions; clear explanations
CS50W (Harvard)	Web programming with Python and JavaScript	Full transcripts; high quality
JavaScript: Understanding the Weird Parts (Udemy)	Deep JS internals	Captions

JavaScript-Specific Accessibility Tools

Tool	Purpose	Notes
ESLint	Code quality and error prevention	Command-line; fully accessible
Prettier	Consistent code formatting	Run on save; no screen interaction needed
Jest	Testing framework	Accessible output; `--verbose` flag
jshint.com	Online JS linter	Plain HTML; basic linting
node-inspect	Node.js debugger	Terminal-based debugging
0x	Node.js profiler	Terminal output
nvm	Node version manager	Command-line; accessible

Communities

Community	Relevance
Node.js Community (nodejs.org/community)	Official Node.js community resources
JavaScript Weekly (newsletter)	Weekly digest of JS news; plain HTML email
Stack Overflow JavaScript	Q&A community; plain HTML; excellent accessibility

Community	Relevance
Dev.to	Developer blog platform; reasonable accessibility
GitHub Discussions on popular JS projects	Community support; text-based
NFB Jernigan Institute	Resources for blind STEM students
SIGACCESS	Research on accessible computing including programming

---

19. Quick Reference Sheets

JavaScript vs Python vs Java at a Glance

Concept	Python	Java	JavaScript
Print	`print("x")`	`System.out.println("x");`	`console.log("x")`
Variable	`x = 5`	`int x = 5;`	`const x = 5;`
String format	`f"Hi {name}"`	`String.format("Hi %s", name)`	`` `Hi ${name}` ``
User input	`input("? ")`	`scanner.nextLine()`	`await askQuestion("? ")`
Array/List	`[1, 2, 3]`	`int[] arr = {1,2,3}`	`[1, 2, 3]`
Dictionary/Map	`{"a": 1}`	`new HashMap<>()`	`{ a: 1 }`
Function	`def f(x):`	`public int f(int x) {`	`const f = x => ...`
Class	`class Dog:`	`public class Dog {`	`class Dog {`
Exception	`try/except`	`try/catch`	`try/catch`
Null	`None`	`null`	`null/undefined`
Boolean	`True/False`	`true/false`	`true/false`
Length	`len(x)`	`x.length`	`x.length`
For each	`for x in list:`	`for (T x : list) {`	`for (const x of list) {`
Import	`import math`	`import java.util.*;`	`const x = require('./x')`

```
// Standard separator
console.log("-".repeat(40));

// Section header
console.log(`\n=== ${title.toUpperCase()} ===`);
```

```
// Numbered list with details
items.forEach((item, i) => {
  console.log(`${i + 1}. ${item.name}`);
  console.log(`   ${item.detail}`);
});
console.log(`Total: ${items.length}`);

// Menu
const MENU = `
--- Main Menu ---
1. Add item
2. View all items
3. Search
4. Edit item
5. Delete item
6. Save
7. Quit
-----------------`;

// Confirmation
console.log(`[x] Added: "${name}"`);
console.log(`[x] Deleted: "${name}"`);
console.log(`[x] Saved to ${filename}`);

// Error messages
console.log(`Error: Name cannot be empty. Please try again.`);
console.log(`Error: "${filename}" not found. Starting with empty data.`);
console.log(`Error: Task number must be between 1 and ${count}.`);
```

Accessible Output Patterns

Common JavaScript Error Quick Reference

Error	Common Cause	First Check
`ReferenceError: x is not defined`	Typo or not declared	Spelling; scope
`TypeError: Cannot read property of undefined`	Accessing property on `undefined`	Which variable is undefined; optional chaining
`TypeError: x is not a function`	Calling non-function	`typeof x` before calling
`SyntaxError: Unexpected token`	Syntax mistake	Line number; check brackets, commas
`SyntaxError: Cannot use import statement`	ES module syntax in CommonJS	Use `require()` or add `"type": "module"`

Error	Common Cause	First Check
`Error: ENOENT: no such file or directory`	File not found	`console.log(__dirname)` and check path
`Error: EACCES: permission denied`	No write permission	Check file permissions
`RangeError: Maximum call stack size exceeded`	Infinite recursion	Check recursive function's base case
`UnhandledPromiseRejection`	Missing `await` or `.catch()`	Add `try/catch` around `await` calls
`NaN` (not an error, but wrong value)	Arithmetic on non-number	`console.log(typeof variable)` before math

```
// utils/input.js
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

/**
 * Prompts the user and returns their response as a string.
 * @param {string} prompt - The question to display
 * @returns {Promise<string>} The user's trimmed response
 */
function askQuestion(prompt) {
  return new Promise((resolve) => {
    rl.question(prompt, (answer) => resolve(answer));
  });
}

/**
 * Prompts until a valid number is entered.
 * @param {string} prompt - The question to display
 * @returns {Promise<number>} A valid number
 */
async function askNumber(prompt) {
  while (true) {
    const input = await askQuestion(prompt);
    const num = Number(input.trim());
    if (!isNaN(num)) return num;
    console.log("Invalid input: please enter a number.");
  }
}
```

```
/**
 * Closes the readline interface. Call at the end of every program.
 */
function close() {
  rl.close();
}

module.exports = { askQuestion, askNumber, close };
```

Input Helper Module (Copy into every project)

Unit 0: Setting Up Your JavaScript Development Environment

Lesson 0.1: Installing Node.js

Node.js lets you run JavaScript on your computer (outside a browser) using the terminal.

Installing Node.js Download **Node.js 18 LTS** from https://nodejs.org/.

Windows

1. Download the `.msi` installer.
2. **CRUCIAL STEP:** Ensure "Add to PATH" is checked during installation.
3. Activate "Install" and "Finish".

macOS

1. Download the `.pkg` installer and follow the prompts.
2. Or use Homebrew: `brew install node`

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install nodejs
```

Linux (Debian/Ubuntu)

```
node --version      # Should show v18.x.x or later
npm --version       # Should show npm version
```

Verify Installation

Skills Checklist

☐ Running `node --version` shows the version number.
☐ Running `npm --version` shows the npm version.

Lesson 0.2: Setting Up VS Code

1. Install VS Code from https://code.visualstudio.com/.
2. Press `Ctrl + Shift + X` to open Extensions. Install:
   - **JavaScript (ES6) code snippets** — Handy code shortcuts.
   - **ESLint** — Catches common errors.
3. Enable Screen Reader Mode: `Ctrl + Shift + P` -> "Toggle Screen Reader Optimized".

VS Code Shortcuts for JavaScript

Action	Shortcut
Run file (via terminal)	`Ctrl + \`` then`node filename.js`| | `Command Palette` |Ctrl + Shift + P| | `Problems View` |Ctrl + Shift + M| | `Next Error` |F8| | `Format Code` |Shift + Alt + F| | `Go to Definition` |F12‘

Lesson 0.3: Your First JavaScript File

Hello World Create `hello.js`:

```
console.log("Hello, World!");
console.log("Welcome to JavaScript!");
```

Run it:

```
node hello.js
```

The Node.js REPL For quick experiments, open an interactive session:

```
node
```

Type JavaScript directly and press Enter to execute. Type `.exit` to quit.

Key Differences from Python

Python	JavaScript
`print()`	`console.log()`
`#` for comments	`//` for single-line, `/* */` for multi-line
No semicolons required	Semicolons optional but recommended
Indentation defines blocks	Curly braces `{}` define blocks
`def` for functions	`function` or `=>` for functions
`True`/`False`	`true`/`false` (lowercase)
`None`	`null` and `undefined`
`len(x)`	`x.length`

Skills Checklist

☐ I can create a `.js` file and run it with `node filename.js`.
☐ I can use the Node.js REPL for quick experiments.

Unit 1: Foundations — Getting Started with JavaScript

Overview

This unit covers JavaScript fundamentals: variables, data types, and basic output.

Lesson 1.1: Welcome to Computer Science

(See Python Unit 1.1 for conceptual content — hardware, software, algorithms, and problem-solving are language-agnostic.)

JavaScript's Role JavaScript was created by Brendan Eich at Netscape in 1995. Originally just for web browsers, it now runs everywhere thanks to Node.js (2009).

Lesson 1.2: Variables and Output

```
console.log("Hello, World!");
console.log(42);
console.log(true);
console.log("Value:", 42, "and", "text");
```

`console.log()`

Declaring Variables JavaScript has three ways to declare variables:

Keyword	Scope	Reassignable	Use When
`const`	Block	No	Value won't change (default choice)
`let`	Block	Yes	Value will change
`var`	Function	Yes	Legacy — avoid in modern code

```
const PI = 3.14159; // Cannot be reassigned
let score = 100; // Can change
score = 150; // OK

// var is function-scoped (legacy)
var oldWay = "avoid this";
```

Best practice: Use `const` by default; use `let` only when you need to reassign.

```
const name = "Alice";
const age = 15;

// Template literal (backticks --- preferred)
console.log(`Hello, ${name}! You are ${age} years old.`);

// Concatenation (older style)
console.log("Hello, " + name + "! You are " + age + " years old.");
```

Template Literals

Skills Checklist

☐ I can use `console.log()` to display output.
☐ I can declare variables using `const` and `let`.
☐ I can use template literals with `${}`.

Lesson 1.3: Data Types

```
// Number (integers and floats use the same type)
let integer = 42;
let decimal = 3.14;
let negative = -10;

// String
let name = "Alice";
let greeting = "Hello";
let multiLine = `Line 1
Line 2`;

// Boolean
let isRaining = false;
let isLoggedIn = true;

// null (intentional absence of value)
let emptyValue = null;

// undefined (value not assigned)
let notAssigned;
console.log(notAssigned); // undefined
```

```
// Check type
console.log(typeof 42); // "number"
console.log(typeof "hello"); // "string"
console.log(typeof true); // "boolean"
console.log(typeof null); // "object" (known quirk!)
console.log(typeof undefined); // "undefined"
```

Primitive Types

```
console.log(5 + 3); // 8
console.log(10 - 4); // 6
console.log(7 * 6); // 42
console.log(15 / 4); // 3.75  (always decimal, unlike Python 2 or Java)
console.log(10 % 3); // 1  (remainder)
console.log(2 ** 10); // 1024  (exponentiation)

// Math.floor for integer division equivalent
console.log(Math.floor(15 / 4)); // 3
```

Arithmetic Operators

```
// String to Number
let str = "123";
let num = Number(str); // 123
let numParsed = parseInt(str); // 123
let floatParsed = parseFloat("3.14"); // 3.14

// Number to String
let n = 42;
let s = String(n); // "42"
let s2 = n.toString(); // "42"
let s3 = `${n}`; // "42" (template literal)

// Checking
console.log(isNaN("abc")); // true --- "not a number"
console.log(isNaN(42)); // false
console.log(Number("abc")); // NaN
```

Type Conversion

```
const MAX_SIZE = 100;
const APP_NAME = "My App";
const PI = Math.PI; // Use Math.PI --- already defined
```

Constants

Examples **Example 1.3.1: Personal Information**

```
const myName = "Jamie";
const favoriteNumber = 7;
const heightInMeters = 1.75;
const likesPizza = true;

console.log(`Name: ${myName}`);
console.log(`Favorite number: ${favoriteNumber}`);
console.log(`Height: ${heightInMeters} meters`);
console.log(`Likes pizza: ${likesPizza}`);
```

Example 1.3.2: Simple Math

```
const num1 = 15;
const num2 = 7;

console.log(`Sum: ${num1 + num2}`);
console.log(`Difference: ${num1 - num2}`);
console.log(`Product: ${num1 * num2}`);
console.log(`Quotient: ${(num1 / num2).toFixed(2)}`);
console.log(`Remainder: ${num1 % num2}`);
```

Activities **Activity 1.3.1: Area Calculator**

```
const length = 12;
const width = 5;
const height = 8;

const rectangleArea = length * width;
const boxVolume = length * width * height;
const circleArea = Math.PI * (length / 2) ** 2;

console.log(`Rectangle area: ${rectangleArea}`);
console.log(`Box volume: ${boxVolume}`);
console.log(`Circle area: ${circleArea.toFixed(2)}`);
```

Skills Checklist

- ☐ I can identify JavaScript's primitive data types.
- ☐ I can perform arithmetic operations in JavaScript.
- ☐ I understand that JavaScript's `/` always gives a decimal result.
- ☐ I can convert between types using `Number()`, `String()`, `parseInt()`.
- ☐ I understand `const` vs. `let`.

Unit 2: Numbers, Input, and the Math Object

Lesson 2.1: The Math Object

JavaScript's built-in `Math` object provides mathematical functions and constants:

```
// Constants
console.log(Math.PI); // 3.14159265358979...
console.log(Math.E); // 2.71828...

// Functions
console.log(Math.sqrt(25)); // 5
console.log(Math.pow(2, 10)); // 1024
console.log(Math.abs(-7.5)); // 7.5
console.log(Math.ceil(4.2)); // 5
console.log(Math.floor(4.8)); // 4
console.log(Math.round(4.5)); // 5
console.log(Math.round(4.4)); // 4
console.log(Math.max(3, 7, 1, 9, 5)); // 9
console.log(Math.min(3, 7, 1, 9, 5)); // 1

// Random number between 0 (inclusive) and 1 (exclusive)
console.log(Math.random());

// Random integer between min and max (inclusive)
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
console.log(randomInt(1, 6)); // Simulates a dice roll
```

Lesson 2.2: Getting User Input in Node.js

Getting user input in Node.js requires the built-in `readline` module.

```

const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

rl.question("What is your name? ", (name) => {
  console.log(`Hello, ${name}!`);
  rl.close();
});
```

Using `readline` (Standard Library)

```
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(prompt) {
  return new Promise((resolve) => {
    rl.question(prompt, (answer) => {
      resolve(answer);
    });
  });
}

async function main() {
  const name = await askQuestion("What is your name? ");
  const ageStr = await askQuestion("How old are you? ");
  const age = parseInt(ageStr);

  console.log(`Hello, ${name}!`);
  console.log(`Next year you will be ${age + 1} years old.`);

  rl.close();
}

main();
```

Promisified Input (Modern Approach)

Helper Module for Input (Reusable) Create `input.js` once and reuse across projects:

```
// input.js --- save this file in your project
const readline = require("readline");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

function askQuestion(prompt) {
  return new Promise((resolve) => {
    rl.question(prompt, resolve);
  });
}
```

```
function close() {
  rl.close();
}

module.exports = { askQuestion, close };
```

Usage in your main file:

```
// main.js
const { askQuestion, close } = require("./input");

async function main() {
  const name = await askQuestion("Enter your name: ");
  const numStr = await askQuestion("Enter a number: ");
  const num = parseFloat(numStr);

  console.log(`Hello, ${name}!`);
  console.log(`Your number squared is: ${num ** 2}`);

  close();
}

main();
```

Examples **Example 2.2.1: Temperature Converter**

```
const { askQuestion, close } = require("./input");

async function main() {
  const fahrenheitStr = await askQuestion("Enter temperature in Fahrenheit: ");
  const fahrenheit = parseFloat(fahrenheitStr);

  const celsius = ((fahrenheit - 32) * 5) / 9;

  console.log(`${fahrenheit}°F is ${celsius.toFixed(1)}°C`);
  close();
}

main();
```

Example 2.2.2: Circle Calculator

```
const { askQuestion, close } = require("./input");

async function main() {
  const radiusStr = await askQuestion("Enter the radius: ");
  const radius = parseFloat(radiusStr);

  const area = Math.PI * radius ** 2;
  const circumference = 2 * Math.PI * radius;
```

```
  console.log(`Area: ${area.toFixed(2)}`);
  console.log(`Circumference: ${circumference.toFixed(2)}`);
  close();
}

main();
```

Skills Checklist

☐ I can use `Math` object methods and constants.
☐ I can get user input using the `readline` module.
☐ I understand `async`/`await` for asynchronous input.
☐ I can convert string input to numbers using `parseInt()` and `parseFloat()`.

Unit 3: Control Flow — Decisions and Loops

Lesson 3.1: Boolean Expressions and `if` Statements

Comparison Operators

Operator	Meaning	Note
`===`	Strict equal	Checks value AND type (preferred)
`!==`	Strict not equal	Checks value AND type (preferred)
`==`	Loose equal	Type coercion (avoid)
`!=`	Loose not equal	Type coercion (avoid)
`<`	Less than	
`>`	Greater than	
`<=`	Less than or equal	
`>=`	Greater than or equal	

Always use `===` and `!==` — `==` does unexpected type conversions.

Logical Operators

Operator	Meaning	Example
`&&`	AND	`age >= 18 && hasId`
`||`	OR	`day === 'Sat' || day === 'Sun'`
`!`	NOT	`!isRaining`

```
const score = 85;

if (score >= 90) {
  console.log("Grade: A");
} else if (score >= 80) {
  console.log("Grade: B");
} else if (score >= 70) {
  console.log("Grade: C");
} else {
  console.log("Grade: F");
}
```

if-else if-else

```
const age = 20;
const status = age >= 18 ? "adult" : "minor";
console.log(status); // "adult"
```

Ternary Operator

```
const day = "Wednesday";
switch (day) {
  case "Monday":
  case "Tuesday":
  case "Wednesday":
  case "Thursday":
  case "Friday":
    console.log("Weekday");
    break;
  case "Saturday":
  case "Sunday":
    console.log("Weekend");
    break;
  default:
    console.log("Unknown day");
}
```

`switch` Statement

Truthy and Falsy Values In JavaScript, the following values are **falsy** (evaluate to `false` in conditions):

- `false`, `0`, `""` (empty string), `null`, `undefined`, `NaN`

Everything else is **truthy**.

```
const name = "";
if (name) {
  console.log("Name is set");
} else {
  console.log("Name is empty"); // This runs
}
```

Examples **Example 3.1.1: Grade Calculator**

```
const { askQuestion, close } = require("./input");

async function main() {
  const scoreStr = await askQuestion("Enter your score (0-100): ");
  const score = parseInt(scoreStr);

  let grade;
  if (score >= 90) grade = "A";
  else if (score >= 80) grade = "B";
  else if (score >= 70) grade = "C";
  else if (score >= 60) grade = "D";
  else grade = "F";

  console.log(`Your grade: ${grade}`);
  close();
}

main();
```

Skills Checklist

☐ I use `===` and `!==` for comparisons in JavaScript.
☐ I can write `if`, `else if`, and `else` statements.
☐ I understand truthy and falsy values.
☐ I can write a `switch` statement.

Lesson 3.2: `for` Loops

```
// Basic for loop
for (let i = 0; i < 5; i++) {
  console.log(`i = ${i}`);
}

// Counting down
for (let i = 10; i > 0; i--) {
  console.log(i);
```

```
}
console.log("Blast off!");

// for...of --- iterate over array elements (like Python's for-in)
const fruits = ["apple", "banana", "cherry"];
for (const fruit of fruits) {
  console.log(`I like ${fruit}.`);
}

// for...in --- iterate over object keys
const person = { name: "Alice", age: 15, city: "Portland" };
for (const key in person) {
  console.log(`${key}: ${person[key]}`);
}
```

```
const numbers = [1, 2, 3, 4, 5];

numbers.forEach((num) => {
  console.log(num * 2);
});

// With index
numbers.forEach((num, index) => {
  console.log(`Index ${index}: ${num}`);
});
```

Array.forEach()

Examples **Example 3.2.1: Multiplication Table**

```
const n = 5;
console.log(`Multiplication table for ${n}:`);
for (let i = 1; i <= 10; i++) {
  console.log(`${n} x ${i} = ${n * i}`);
}
```

Skills Checklist

☐ I can write a `for` loop with initialization, condition, and update.
☐ I can use `for...of` to iterate over arrays.
☐ I can use `for...in` to iterate over object keys.
☐ I can use `Array.forEach()`.

Lesson 3.3: `while` Loops

```
let count = 1;
while (count <= 5) {
  console.log(`Count: ${count}`);
  count++;
}

// do-while --- runs at least once
let num;
do {
  num = Math.floor(Math.random() * 10);
  console.log(`Rolled: ${num}`);
} while (num < 5);
```

```
// break --- exit entirely
for (let i = 0; i < 10; i++) {
  if (i === 5) break;
  console.log(i); // Prints 0-4
}

// continue --- skip iteration
for (let i = 0; i < 10; i++) {
  if (i % 2 === 0) continue;
  console.log(i); // Prints 1, 3, 5, 7, 9
}
```

`break` and `continue`

Skills Checklist

☐ I can write `while` and `do-while` loops.
☐ I can use `break` and `continue` appropriately.

Unit 4: Data Structures — Arrays and Objects

Lesson 4.1: Arrays

```
const colors = ["red", "blue", "green", "yellow"];

// Access
console.log(colors[0]); // "red"
```

```
console.log(colors[colors.length - 1]); // "yellow"

// Modify
colors[1] = "purple";

// Check
console.log(colors.length); // 4
console.log(colors.includes("red")); // true
console.log(colors.indexOf("green")); // 2
```

Creating and Accessing Arrays

```
const fruits = ["apple", "banana"];

// Adding
fruits.push("cherry"); // Add to end -> ["apple", "banana", "cherry"]
fruits.unshift("avocado"); // Add to start -> ["avocado", "apple", "banana", "cherry"]

// Removing
const last = fruits.pop(); // Remove and return last
const first = fruits.shift(); // Remove and return first

// Slicing (non-destructive)
const sub = fruits.slice(1, 3); // Returns new array, original unchanged

// Splicing (destructive --- modifies original)
fruits.splice(1, 1); // Remove 1 element at index 1
fruits.splice(1, 0, "mango"); // Insert "mango" at index 1

// Joining
const str = fruits.join(", "); // "apple, mango, cherry"

// Sorting
const nums = [3, 1, 4, 1, 5, 9, 2, 6];
nums.sort((a, b) => a - b); // Ascending: [1, 1, 2, 3, 4, 5, 6, 9]
nums.sort((a, b) => b - a); // Descending

// Reversing
const reversed = [...fruits].reverse(); // Use spread to avoid mutating original

// Concatenating
const combined = fruits.concat(nums);
```

Array Methods

Functional Array Methods These are powerful for data transformation:

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// map --- transform each element, returns new array
const doubled = numbers.map((n) => n * 2);
console.log(doubled); // [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

// filter --- keep elements matching condition
const evens = numbers.filter((n) => n % 2 === 0);
console.log(evens); // [2, 4, 6, 8, 10]

// reduce --- combine elements into single value
const sum = numbers.reduce((accumulator, current) => accumulator + current, 0);
console.log(sum); // 55

// find --- return first matching element
const firstOver5 = numbers.find((n) => n > 5);
console.log(firstOver5); // 6

// some --- true if any element matches
const hasEven = numbers.some((n) => n % 2 === 0);
console.log(hasEven); // true

// every --- true if all elements match
const allPositive = numbers.every((n) => n > 0);
console.log(allPositive); // true

// Chaining
const result = numbers
  .filter((n) => n % 2 === 0)
  .map((n) => n ** 2)
  .reduce((sum, n) => sum + n, 0);
console.log(result); // 220  (4+16+36+64+100)
```

Examples **Example 4.1.1: Array Analysis**

```
const scores = [85, 92, 78, 95, 88, 72, 90, 83];

const average = scores.reduce((sum, s) => sum + s, 0) / scores.length;
const max = Math.max(...scores);
const min = Math.min(...scores);
const passing = scores.filter((s) => s >= 80);

console.log(`Scores: ${scores.join(", ")}`);
console.log(`Average: ${average.toFixed(1)}`);
console.log(`Highest: ${max}`);
console.log(`Lowest: ${min}`);
console.log(`Passing (>=80): ${passing.length} students`);
```

Skills Checklist

- ☐ I can create, access, and modify arrays.
- ☐ I can use `push`, `pop`, `shift`, `unshift`, `splice`, and `slice`.
- ☐ I can use `map`, `filter`, and `reduce`.
- ☐ I can sort arrays using a comparator function.

Lesson 4.2: Objects

Creating and Using Objects Objects in JavaScript are like Python dictionaries and Java Maps combined with class instances.

```
// Object literal
const studentProfile = {
  name: "Emily",
  age: 13,
  grade: "7th",
  favoriteSubject: "Science",
};

// Access properties
console.log(studentProfile.name); // "Emily" (dot notation)
console.log(studentProfile["age"]); // 13 (bracket notation)

// Add/modify
studentProfile.city = "Springfield";
studentProfile.age = 14;

// Delete
delete studentProfile.grade;

// Check if property exists
console.log("name" in studentProfile); // true
console.log(studentProfile.hasOwnProperty("city")); // true

// Get keys, values, entries
console.log(Object.keys(studentProfile));
console.log(Object.values(studentProfile));
console.log(Object.entries(studentProfile));
```

```
const inventory = { apples: 50, bananas: 30, oranges: 40 };

// for...in
for (const key in inventory) {
  console.log(`${key}: ${inventory[key]}`);
}

// Object.entries() with destructuring
```

```
for (const [key, value] of Object.entries(inventory)) {
  console.log(`${key}: ${value} in stock`);
}
```

Iterating Over Objects

```
const library = {
  name: "City Library",
  location: "Downtown",
  books: [
    { title: "Dune", author: "Frank Herbert", available: true },
    { title: "1984", author: "George Orwell", available: false },
  ],
};

// Access nested data
console.log(library.books[0].title); // "Dune"
console.log(library.books[1].author); // "George Orwell"

// Filter available books
const available = library.books.filter((book) => book.available);
console.log(`Available: ${available.map((b) => b.title).join(", ")}`);
```

Nested Objects and Arrays

```
// Object destructuring
const { name, age } = studentProfile;
console.log(name, age); // "Emily" 13

// With renaming
const { name: studentName, favoriteSubject: subject } = studentProfile;

// Array destructuring
const [first, second, ...rest] = [1, 2, 3, 4, 5];
console.log(first); // 1
console.log(rest); // [3, 4, 5]

// In function parameters
function printStudent({ name, age, grade }) {
  console.log(`${name} | Age: ${age} | Grade: ${grade}`);
}
printStudent(studentProfile);
```

Destructuring

```
// Spread --- expand array or object
const arr1 = [1, 2, 3];
const arr2 = [4, 5, 6];
const combined = [...arr1, ...arr2]; // [1, 2, 3, 4, 5, 6]

const obj1 = { a: 1, b: 2 };
const obj2 = { c: 3, d: 4 };
const merged = { ...obj1, ...obj2 }; // { a: 1, b: 2, c: 3, d: 4 }

// Copy without mutating
const originalArr = [1, 2, 3];
const copyArr = [...originalArr];

const originalObj = { name: "Alice" };
const copyObj = { ...originalObj };
```

Spread and Rest Operators

Skills Checklist

- ☐ I can create, access, modify, and delete object properties.
- ☐ I can iterate over objects using `for...in` and `Object.entries()`.
- ☐ I can work with nested objects and arrays.
- ☐ I can use destructuring assignment.
- ☐ I can use spread and rest operators.

Unit 5: Strings and String Methods

Lesson 5.1: Working with Strings

```
const name = "Alice Smith";

// Properties
console.log(name.length); // 11

// Accessing characters
console.log(name[0]); // "A"
console.log(name.charAt(0)); // "A"
console.log(name[name.length - 1]); // "h"

// Substrings
console.log(name.slice(6)); // "Smith"
console.log(name.slice(0, 5)); // "Alice"
console.log(name.substring(6, 11)); // "Smith"

// Case
console.log(name.toUpperCase()); // "ALICE SMITH"
console.log(name.toLowerCase()); // "alice smith"

// Trimming
const padded = "  hello  ";
console.log(padded.trim()); // "hello"
console.log(padded.trimStart()); // "hello  "
console.log(padded.trimEnd()); // "  hello"

// Searching
console.log(name.indexOf("Smith")); // 6
console.log(name.lastIndexOf("l")); // 3
console.log(name.includes("Alice")); // true
console.log(name.startsWith("Ali")); // true
console.log(name.endsWith("ith")); // true

// Replacing
const result = name.replace("Alice", "Bob"); // "Bob Smith"
const allReplaced = "a,b,a,c,a".replaceAll("a", "x"); // "x,b,x,c,x"

// Splitting
const sentence = "the quick brown fox";
const words = sentence.split(" ");
console.log(words); // ["the", "quick", "brown", "fox"]

const csv = "Alice,15,Portland";
const parts = csv.split(",");
console.log(parts); // ["Alice", "15", "Portland"]
```

```
// Repeating and Padding
console.log("ha".repeat(3)); // "hahaha"
console.log("5".padStart(3, "0")); // "005"
console.log("5".padEnd(3, "0")); // "500"

// Template literals (multi-line)
const multiLine = `
Name: ${name}
Length: ${name.length}
Uppercase: ${name.toUpperCase()}
`;
console.log(multiLine);
```

```
const text = "Hello World! Hello JavaScript!";

// Test if pattern exists
console.log(/hello/i.test(text)); // true (i = case-insensitive)

// Find matches
const matches = text.match(/Hello/g); // g = global (find all)
console.log(matches); // ["Hello", "Hello"]

// Replace with regex
const cleaned = text.replace(/[!]/g, "."); // "Hello World. Hello JavaScript."

// Split with regex
const words2 = "one  two   three".split(/\s+/);
console.log(words2); // ["one", "two", "three"]
```

Regular Expressions (Basics)

Skills Checklist

☐ I can use common `String` methods in JavaScript.
☐ I can use template literals for multi-line strings and interpolation.
☐ I can split strings into arrays and join arrays into strings.
☐ I understand basic regular expressions.

Unit 6: Functions

Lesson 6.1: Defining Functions

JavaScript has multiple ways to define functions:

```
function greetUser() {
  console.log("Hello, JavaScript programmer!");
}

greetUser();
greetUser();
```

Function Declaration

```
const greetByName = function (name) {
  console.log(`Hello, ${name}!`);
};

greetByName("Alice");
```

Function Expression

```
// Full arrow function
const add = (a, b) => {
  return a + b;
};

// Concise body (implicit return for single expressions)
const addShort = (a, b) => a + b;

// Single parameter (no parentheses needed)
const double = (n) => n * 2;

// No parameters
const sayHello = () => console.log("Hello!");

console.log(add(5, 3)); // 8
console.log(addShort(5, 3)); // 8
console.log(double(7)); // 14
sayHello();
```

Arrow Functions (ES6) — Modern Preferred Style

```
function makeShirt(size, message = "I love JavaScript") {
  console.log(`Making a ${size}-sized shirt: '${message}'`);
}
```

```
makeShirt("large"); // Uses default
makeShirt("medium", "Hello World"); // Custom message
```

Default Parameters

```
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}

console.log(sum(1, 2, 3)); // 6
console.log(sum(10, 20, 30, 40)); // 100
```

Rest Parameters

Examples **Example 6.1.1: Calculator Functions**

```
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;
const multiply = (a, b) => a * b;
const divide = (a, b) => {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
};

const circleArea = (radius) => Math.PI * radius ** 2;
const getGrade = (score) => {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  if (score >= 60) return "D";
  return "F";
};

console.log(add(5, 3));
console.log(circleArea(5).toFixed(2));
console.log(getGrade(85));
```

Example 6.1.2: Higher-Order Functions

```
// Functions that take or return other functions

// Map using our own function
function doubleAll(numbers) {
  return numbers.map((n) => n * 2);
}

// Function that returns a function (closure)
function makeAdder(x) {
```

```
  return (y) => x + y;
}

const addFive = makeAdder(5);
const addTen = makeAdder(10);

console.log(addFive(3)); // 8
console.log(addTen(3)); // 13
console.log(doubleAll([1, 2, 3, 4, 5]));
```

```
function makeCounter() {
  let count = 0; // Private variable

  return {
    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count,
  };
}

const counter = makeCounter();
counter.increment();
counter.increment();
counter.increment();
counter.decrement();
console.log(counter.getCount()); // 2
```

Closures

Skills Checklist

- ☐ I can define functions using declarations, expressions, and arrow syntax.
- ☐ I can use default and rest parameters.
- ☐ I understand closures and how they capture variables.
- ☐ I can pass functions as arguments and return them.

Unit 7: Classes and Object-Oriented Programming

Lesson 7.1: Classes in JavaScript

JavaScript uses prototype-based inheritance, but ES6 `class` syntax provides a familiar OOP style.

```
class Dog {
  // Instance variables declared in constructor
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  // Methods
  sit() {
    console.log(`${this.name} is now sitting.`);
  }

  rollOver() {
    console.log(`${this.name} rolled over!`);
  }

  getOlder(years) {
    this.age += years;
    console.log(`${this.name} is now ${this.age} years old.`);
  }

  // Getter
  get info() {
    return `${this.name} (${this.age} years old)`;
  }

  // toString equivalent
  toString() {
    return `Dog { name: '${this.name}', age: ${this.age} }`;
  }
}

// Create instances
const myDog = new Dog("Buddy", 3);
const yourDog = new Dog("Lucy", 1);

myDog.sit();
myDog.getOlder(2);
console.log(myDog.info); // Uses getter
console.log(myDog.toString());
```

Class Syntax

```
class BankAccount {
  #balance; // Private field --- only accessible within the class

  constructor(initialBalance) {
```

```
    this.#balance = initialBalance;
  }

  deposit(amount) {
    if (amount > 0) {
      this.#balance += amount;
      console.log(`Deposited $${amount}. New balance: $${this.#balance}`);
    }
  }

  withdraw(amount) {
    if (amount > this.#balance) {
      console.log("Insufficient funds!");
    } else {
      this.#balance -= amount;
      console.log(`Withdrew $${amount}. New balance: $${this.#balance}`);
    }
  }

  get balance() {
    return this.#balance;
  }
}

const account = new BankAccount(1000);
account.deposit(500);
account.withdraw(200);
console.log(`Balance: $${account.balance}`);
```

Private Fields (ES2022)

```
class MathHelper {
  static add(a, b) {
    return a + b;
  }
  static multiply(a, b) {
    return a * b;
  }
  static PI = 3.14159265358979;
}

// Call without creating an instance
console.log(MathHelper.add(5, 3));
console.log(MathHelper.PI);
```

Static Methods

Lesson 7.2: Inheritance

```
class Vehicle {
  constructor(make, model, year) {
    this.make = make;
    this.model = model;
    this.year = year;
    this.odometer = 0;
  }

  getDescriptiveName() {
    return `${this.year} ${this.make} ${this.model}`;
  }

  readOdometer() {
    console.log(`This vehicle has ${this.odometer} miles on it.`);
  }
}

class Car extends Vehicle {
  constructor(make, model, year) {
    super(make, model, year); // Call parent constructor
    this.numDoors = 4;
  }

  readOdometer() {
    // Override parent method
    console.log(`This car has ${this.odometer} miles on it.`);
  }
}

class ElectricCar extends Car {
  #batterySize;

  constructor(make, model, year) {
    super(make, model, year);
    this.#batterySize = 75;
  }

  describeBattery() {
    console.log(`This car has a ${this.#batterySize}-kWh battery.`);
  }
}

const myEV = new ElectricCar("Ford", "Mustang Mach-E", 2023);
console.log(myEV.getDescriptiveName()); // Inherited from Vehicle
myEV.readOdometer(); // Overridden in Car
myEV.describeBattery(); // Defined in ElectricCar
```

Skills Checklist

- ☐ I can define a JavaScript class with a constructor and methods.
- ☐ I can use `extends` to create a subclass.
- ☐ I can use `super()` to call the parent constructor.
- ☐ I can use `#` for private fields.
- ☐ I can define static methods and properties.

Unit 8: Files, Modules, and Error Handling

Lesson 8.1: Reading and Writing Files

```
const fs = require("fs");
const path = require("path");

// Synchronous reading (simple but blocks execution)
try {
  const content = fs.readFileSync("my_notes.txt", "utf8");
  console.log(content);
} catch (err) {
  console.log(`Error: ${err.message}`);
}

// Reading line by line
const lines = content.split("\n");
lines.forEach((line, index) => {
  console.log(`Line ${index + 1}: ${line}`);
});

// Writing (overwrites existing content)
try {
  fs.writeFileSync("output.txt", "Hello, Node.js file I/O!\n");
  console.log("File written successfully.");
} catch (err) {
  console.log(`Error writing file: ${err.message}`);
}

// Appending
try {
  fs.appendFileSync("output.txt", "This line is appended.\n");
  console.log("Line appended.");
} catch (err) {
  console.log(`Error appending: ${err.message}`);
}

// Check if file exists
if (fs.existsSync("my_notes.txt")) {
  console.log("File exists.");
}
```

```
const fs = require("fs").promises; // Promise-based fs

async function readFile() {
  try {
    const content = await fs.readFile("my_notes.txt", "utf8");
    console.log(content);
  } catch (err) {
    console.log(`Error reading file: ${err.message}`);
  }
}

async function writeFile() {
  try {
    await fs.writeFile("output.txt", "Async write!\n");
    await fs.appendFile("output.txt", "Appended!\n");
    console.log("Files written.");
  } catch (err) {
    console.log(`Error: ${err.message}`);
  }
}

readFile();
writeFile();
```

Asynchronous File I/O (Modern Approach)

Lesson 8.2: Error Handling

```
// try-catch-finally
try {
  const result = 10 / 0; // Doesn't throw in JS (gives Infinity)
  const parsed = JSON.parse("invalid json"); // This throws
} catch (err) {
  console.log(`Error type: ${err.constructor.name}`);
  console.log(`Error message: ${err.message}`);
} finally {
  console.log("This always runs.");
}

// Throwing custom errors
function divide(a, b) {
  if (b === 0) {
    throw new Error("Division by zero is not allowed.");
  }
  return a / b;
}

try {
```

```
  console.log(divide(10, 2)); // 5
  console.log(divide(10, 0)); // Throws
} catch (err) {
  console.log(`Caught: ${err.message}`);
}
```

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message);
    this.name = "ValidationError";
    this.field = field;
  }
}

class InsufficientFundsError extends Error {
  constructor(needed) {
    super(`Insufficient funds. Need $${needed} more.`);
    this.name = "InsufficientFundsError";
    this.needed = needed;
  }
}

try {
  throw new ValidationError("email", "Invalid email format.");
} catch (err) {
  if (err instanceof ValidationError) {
    console.log(`Validation failed on '${err.field}': ${err.message}`);
  } else {
    throw err; // Re-throw if not our custom error
  }
}
```

Custom Error Classes

Lesson 8.3: JSON in Node.js

```
const fs = require("fs");

// JavaScript object to JSON string
const settings = {
  username: "coder_kid",
  theme: "dark",
  notificationsOn: true,
  favoriteNumbers: [7, 13, 42],
};
```

```
const jsonString = JSON.stringify(settings, null, 2); // Pretty-printed
console.log(jsonString);

// Save to file
fs.writeFileSync("settings.json", jsonString);

// Load from file
try {
  const loaded = JSON.parse(fs.readFileSync("settings.json", "utf8"));
  console.log(`Username: ${loaded.username}`);
  console.log(`Theme: ${loaded.theme}`);
  console.log(`Numbers: ${loaded.favoriteNumbers}`);
} catch (err) {
  if (err instanceof SyntaxError) {
    console.log("Error: JSON file is corrupted.");
  } else if (err.code === "ENOENT") {
    console.log("Error: File not found.");
  } else {
    console.log(`Error: ${err.message}`);
  }
}
```

Skills Checklist

☐ I can read and write files using the `fs` module.
☐ I can use `try-catch-finally` to handle errors.
☐ I can throw custom errors using `Error` or custom classes.
☐ I can serialize/deserialize data using `JSON.stringify` and `JSON.parse`.

Unit 9: Modules, Collections, and Testing

Lesson 9.1: JavaScript Modules

```
// math.js --- exporting
const add = (a, b) => a + b;
const multiply = (a, b) => a * b;
const PI = Math.PI;

module.exports = { add, multiply, PI };

// Or export individually:
// module.exports.add = add;
```

```
// main.js --- importing
const { add, multiply, PI } = require("./math");

console.log(add(5, 3)); // 8
console.log(multiply(4, 6)); // 24
console.log(PI.toFixed(5)); // 3.14159
```

CommonJS (Node.js default)

```
// math.mjs
export const add = (a, b) => a + b;
export const multiply = (a, b) => a * b;
export const PI = Math.PI;
export default function greet(name) {
  return `Hello, ${name}!`;
}
```

```
// main.mjs
import greet, { add, multiply, PI } from "./math.mjs";

console.log(greet("Alice"));
console.log(add(5, 3));
```

ES Modules (Modern — requires `"type": "module"` in package.json)

Lesson 9.2: Maps and Sets

```
const inventory = new Map();

// Adding
inventory.set("apples", 50);
inventory.set("bananas", 30);
inventory.set("oranges", 40);

// Access
console.log(inventory.get("apples")); // 50
console.log(inventory.has("bananas")); // true
console.log(inventory.size); // 3

// Delete
inventory.delete("bananas");

// Iterate
for (const [key, value] of inventory) {
  console.log(`${key}: ${value}`);
}

inventory.forEach((value, key) => {
  console.log(`${key} = ${value}`);
});

console.log([...inventory.keys()]);
console.log([...inventory.values()]);
console.log([...inventory.entries()]);
```

`Map` — *Key-Value (Better than plain objects for maps)*

```
const uniqueWords = new Set();
uniqueWords.add("apple");
uniqueWords.add("banana");
uniqueWords.add("apple"); // Duplicate --- ignored

console.log(uniqueWords.size); // 2
console.log(uniqueWords.has("banana")); // true
uniqueWords.delete("banana");

// Convert array to set (remove duplicates)
const nums = [1, 2, 2, 3, 3, 3, 4];
const unique = [...new Set(nums)];
console.log(unique); // [1, 2, 3, 4]

// Iterate
```

```
for (const item of uniqueWords) {
  console.log(item);
}
```

`Set` — Unique Values

Lesson 9.3: Word Frequency Analysis

```
const fs = require("fs");

function countWords(text) {
  const words = text
    .toLowerCase()
    .replace(/[^a-z\s]/g, "")
    .trim()
    .split(/\s+/)
    .filter((word) => word.length > 0);

  const frequency = new Map();
  for (const word of words) {
    frequency.set(word, (frequency.get(word) || 0) + 1);
  }

  return frequency;
}

const text = "the quick brown fox jumps over the lazy dog the quick brown fox";
const freq = countWords(text);

// Sort by frequency (most common first)
const sorted = [...freq.entries()].sort((a, b) => b[1] - a[1]);

console.log("Word Frequencies:");
for (const [word, count] of sorted) {
  console.log(`'${word}': ${count}`);
}

// Top 5 words
console.log("\nTop 5 words:");
sorted.slice(0, 5).forEach(([word, count]) => {
  console.log(`  ${word}: ${count}`);
});
```

Lesson 9.4: Working with CSV Files

```
const fs = require("fs");

function parseCSV(content) {
  const lines = content.trim().split("\n");
  const headers = lines[0].split(",").map((h) => h.trim());

  return lines.slice(1).map((line) => {
    const values = line.split(",").map((v) => v.trim());
    const record = {};
    headers.forEach((header, index) => {
      record[header] = values[index];
    });
    return record;
  });
}

function generateCSV(records) {
  if (records.length === 0) return "";
  const headers = Object.keys(records[0]);
  const headerLine = headers.join(",");
  const dataLines = records.map((record) =>
    headers.map((h) => record[h]).join(","),
  );
  return [headerLine, ...dataLines].join("\n");
}

// Read and parse CSV
try {
  const content = fs.readFileSync("students.csv", "utf8");
  const students = parseCSV(content);

  console.log(`Loaded ${students.length} students:`);
  students.forEach((student) => {
    console.log(
      `  Name: ${student.name}, Age: ${student.age}, Grade: ${student.grade}`,
    );
  });
} catch (err) {
  if (err.code === "ENOENT") {
    // Create a sample CSV for demonstration
    const sampleData = `name,age,grade
Alice,15,A
Bob,14,B
Charlie,16,A`;
    fs.writeFileSync("students.csv", sampleData);
    console.log("Created sample students.csv");
  }
}
```

Lesson 9.5: Testing with Jest

```
npm init -y
npm install --save-dev jest
```

Setting Up Jest Add to `package.json`:

```
{
  "scripts": {
    "test": "jest"
  }
}
```

```
// math.js
const add = (a, b) => a + b;
const getGrade = (score) => {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  return "F";
};
const circleArea = (radius) => Math.PI * radius ** 2;

module.exports = { add, getGrade, circleArea };
```

```
// math.test.js
const { add, getGrade, circleArea } = require("./math");

describe("add function", () => {
  test("adds two positive numbers", () => {
    expect(add(5, 3)).toBe(8);
  });

  test("adds negative numbers", () => {
    expect(add(-5, -3)).toBe(-8);
  });

  test("adds zero", () => {
    expect(add(0, 7)).toBe(7);
  });
});

describe("getGrade function", () => {
  test("returns A for score >= 90", () => {
    expect(getGrade(95)).toBe("A");
```

```
    expect(getGrade(90)).toBe("A");
  });

  test("returns B for score 80-89", () => {
    expect(getGrade(85)).toBe("B");
  });

  test("returns F for score < 60", () => {
    expect(getGrade(55)).toBe("F");
  });
});

describe("circleArea function", () => {
  test("calculates area of circle", () => {
    expect(circleArea(5)).toBeCloseTo(78.54, 1);
  });
});
```

Writing Tests Run tests:

```
npm test
```

Common Jest Matchers

Matcher	Description
`toBe(value)`	Strict equality (===)
`toEqual(value)`	Deep equality (for objects/arrays)
`toBeCloseTo(num, decimals)`	Floating-point comparison
`toBeTruthy()`	Check if truthy
`toBeFalsy()`	Check if falsy
`toBeNull()`	Check if null
`toBeUndefined()`	Check if undefined
`toContain(item)`	Array/string contains item
`toHaveLength(n)`	Check length
`toThrow(error)`	Check if function throws

Skills Checklist

☐ I can create and import modules using CommonJS.
☐ I can use `Map` and `Set` for collections.
☐ I can write and run Jest tests.
☐ I can use common Jest matchers.

Unit 10: Project-Based Learning

Lesson 10.1: Text-Based Adventure Game

```
class Room {
  constructor(name, description) {
    this.name = name;
    this.description = description;
    this.exits = {};
    this.items = [];
  }

  addExit(direction, room) {
    this.exits[direction.toLowerCase()] = room;
  }

  addItem(item) {
    this.items.push(item);
  }

  describe() {
    let desc = `--- ${this.name} ---\n${this.description}\n`;
    if (this.items.length > 0) {
      desc += `You see: ${this.items.join(", ")}\n`;
    }
    const exitList = Object.keys(this.exits).join(", ");
    desc += `Exits: ${exitList}`;
    return desc;
  }
}

class Player {
  constructor(name, startRoom) {
    this.name = name;
    this.currentRoom = startRoom;
    this.inventory = [];
    this.health = 100;
  }

  move(direction) {
    const nextRoom = this.currentRoom.exits[direction.toLowerCase()];
    if (nextRoom) {
      this.currentRoom = nextRoom;
      return true;
    }
    return false;
  }

  takeItem(itemName) {
    const index = this.currentRoom.items.findIndex(
      (item) => item.toLowerCase() === itemName.toLowerCase(),
    );
    if (index !== -1) {
```

```
      const item = this.currentRoom.items.splice(index, 1)[0];
      this.inventory.push(item);
      return true;
    }
    return false;
  }

  getStatus() {
    const inv = this.inventory.length > 0 ? this.inventory.join(", ") : "empty";
    return `${this.name} | Health: ${this.health} | Inventory: ${inv}`;
  }
}

async function playGame() {
  const { askQuestion, close } = require("./input");

  // Build world
  const cave = new Room("Dark Cave", "You are in a dimly lit cave.");
  const tunnel = new Room(
    "Stone Tunnel",
    "A narrow tunnel runs north and south.",
  );
  const clearing = new Room(
    "Sunny Clearing",
    "A bright clearing with tall trees.",
  );

  cave.addExit("north", tunnel);
  cave.addItem("torch");
  tunnel.addExit("south", cave);
  tunnel.addExit("north", clearing);
  clearing.addExit("south", tunnel);
  clearing.addItem("magic key");

  const player = new Player("Hero", cave);

  console.log("=== Text Adventure ===");
  console.log("Commands: go [direction], take [item], inventory, look, quit\n");

  while (true) {
    console.log(player.currentRoom.describe());
    console.log(player.getStatus());

    const input = (await askQuestion("\n> ")).trim().toLowerCase();
    const [command, ...argParts] = input.split(/\s+/);
    const argument = argParts.join(" ");

    console.log("");

    switch (command) {
      case "go":
```

```
        if (!argument) {
          console.log("Go where?");
        } else if (!player.move(argument)) {
          console.log("You can't go that way.");
        }
        break;
      case "take":
        if (player.takeItem(argument)) {
          console.log(`You picked up the ${argument}.`);
        } else {
          console.log(`There's no ${argument} here.`);
        }
        break;
      case "inventory":
        console.log(`Inventory: ${player.inventory.join(", ") || "empty"}`);
        break;
      case "look":
        // Description already shown at top
        break;
      case "quit":
        console.log("Thanks for playing!");
        close();
        return;
      default:
        console.log(`Unknown command: '${command}'`);
    }
  }
}

playGame();
```

Lesson 10.2: Command-Line To-Do Manager

```
const fs = require("fs");
const { askQuestion, close } = require("./input");

const FILE_PATH = "todos.json";

function loadTodos() {
  try {
    const data = fs.readFileSync(FILE_PATH, "utf8");
    return JSON.parse(data);
  } catch (err) {
    if (err.code === "ENOENT") {
      return []; // File doesn't exist yet
    }
    if (err instanceof SyntaxError) {
      console.log("Warning: Corrupted todos file. Starting fresh.");
```

```
      return [];
    }
    throw err;
  }
}

function saveTodos(todos) {
  fs.writeFileSync(FILE_PATH, JSON.stringify(todos, null, 2));
}

function addTask(todos, description) {
  todos.push({
    description,
    completed: false,
    createdAt: new Date().toISOString(),
  });
  console.log(`Task added: "${description}"`);
}

function viewTasks(todos) {
  if (todos.length === 0) {
    console.log("No tasks in your list.");
    return;
  }
  console.log("--- Your To-Do List ---");
  todos.forEach((task, index) => {
    const status = task.completed ? "[X]" : "[ ]";
    console.log(`${index + 1}. ${status} ${task.description}`);
  });
  console.log("-------------------------");
}

function markComplete(todos, taskNumber) {
  const index = parseInt(taskNumber) - 1;
  if (index >= 0 && index < todos.length) {
    todos[index].completed = true;
    console.log(`Completed: "${todos[index].description}"`);
  } else {
    console.log("Invalid task number.");
  }
}

function deleteTask(todos, taskNumber) {
  const index = parseInt(taskNumber) - 1;
  if (index >= 0 && index < todos.length) {
    const removed = todos.splice(index, 1)[0];
    console.log(`Deleted: "${removed.description}"`);
  } else {
    console.log("Invalid task number.");
  }
}
```

```
async function main() {
  let todos = loadTodos();
  console.log("=== To-Do Manager ===");

  while (true) {
    console.log(
      "\n1. Add Task  2. View Tasks  3. Mark Complete  4. Delete  5. Quit",
    );
    const choice = (await askQuestion("Choice: ")).trim();

    switch (choice) {
      case "1":
        const desc = await askQuestion("Task description: ");
        addTask(todos, desc);
        saveTodos(todos);
        break;
      case "2":
        viewTasks(todos);
        break;
      case "3":
        viewTasks(todos);
        const completeNum = await askQuestion("Task number: ");
        markComplete(todos, completeNum);
        saveTodos(todos);
        break;
      case "4":
        viewTasks(todos);
        const deleteNum = await askQuestion("Task number to delete: ");
        deleteTask(todos, deleteNum);
        saveTodos(todos);
        break;
      case "5":
        console.log("Goodbye!");
        close();
        return;
      default:
        console.log("Invalid choice.");
    }
  }
}

main();
```

Unit 11: Capstone Project — Personal Data Manager in JavaScript

Overview

Build a fully functional, accessible command-line personal data manager using Node.js.

Project Options

- **Contact Manager:** name, phone, email, notes
- **Book Tracker:** title, author, genre, read status
- **Recipe Collection:** name, ingredients, instructions, cuisine type
- **Budget Tracker:** category, amount, date, description

Required Features

1. **Add item** — prompt for all attributes and save.
2. **View all items** — display clearly with numbers.
3. **Search/filter** — find items by keyword.
4. **Edit item** — select and modify specific fields.
5. **Delete item** — remove by number.
6. **Persistence** — save/load to JSON file.

Project Structure

```
my-data-manager/
|--- src/
|   |--- models/
|   |   +--- contact.js      (data model/class)
|   |--- storage/
|   |   +--- dataManager.js  (JSON file I/O)
|   |--- ui/
|   |   +--- consoleUI.js    (menu and display)
|   +--- utils/
|       +--- input.js        (reusable input helper)
|--- data/
|   +--- contacts.json       (data storage)
|--- tests/
|   +--- contact.test.js     (Jest tests)
|--- package.json
|--- README.md
+--- index.js                (entry point)
```

Accessibility Guidelines

1. Every prompt must clearly describe what input is expected.
2. Print a confirmation message after every action.
3. Number all listed items for easy selection.
4. Provide clear, specific error messages.
5. Include a `help` command listing all available options.
6. Use consistent separators (`---`) to group output.

Assessment Criteria

Functionality (60%)

☐ All CRUD operations (Create, Read, Update, Delete) work correctly.

☐ Data persists to JSON between program runs.
☐ Search works by at least one field.
☐ Program handles invalid input without crashing.

Code Quality (20%)

☐ Functions are small with single responsibilities.
☐ Consistent use of `const`/`let`, arrow functions, template literals.
☐ Meaningful names for all variables and functions.
☐ Code is organized into modules.
☐ Includes JSDoc comments for key functions.

Accessibility & UX (15%)

☐ All prompts are descriptive and clear.
☐ Confirmation messages after every action.
☐ Consistent, predictable output format.
☐ Help system available.

Documentation & Tests (5%)

☐ `README.md` with setup and usage instructions.
☐ At least 5 Jest unit tests for key functions.

Deliverables

1. All source code files.
2. `data/` folder with example JSON data.
3. `README.md` including:
    - Installation steps: `npm install`
    - Run instructions: `node index.js`
    - Available commands list.
    - Screen reader usage tips.
    - Reflection on challenges faced.